

Jak działa Internet – HTTP

Po lekturze dwóch poprzednich artykułów tej serii wiemy już, jak skonfigurować własną sieć domową (rozumiemy działanie routerów i switchy) oraz jak uruchomić prosty serwis internetowy stworzony przez kogoś innego (potrafimy przypisać nazwy DNS do konkretnych adresów IP). Teraz nadszedł czas, aby zgłębić „język”, którym posługuje się większość serwisów – protokół HTTP. Dokładne jego zrozumienie otworzy nam drogę do samodzielnej implementacji serwisów, lepszego rozumienia komunikacji internetowej na poziomie aplikacji, a także do efektywnego wykrywania i naprawiania problemów z komunikacją.

I CZYM JEST HTTP?

HTTP (ang. *HyperText Transfer Protocol*) to jeden z protokołów warstwy aplikacji, działający na siódmej warstwie modelu OSI [2]. Został stworzony z myślą o komunikacji między klientami (najczęściej przeglądarkami internetowymi) a serwerami (komputerami przechowującymi dane) [1]. Ta relacja stanowi podstawę najprostszej wersji architektury serwisów internetowych. W kolejnych rozdziałach (Proxy – mitproxy, Cache) omówimy również bardziej zaawansowane elementy tej architektury, jak serwery proxy, a także *cache* [2].

HTTP zazwyczaj korzysta z protokołu TCP do nawiązywania połączenia oraz do zapewnienia integralności przesyłanych danych. Warto wspomnieć, że HTTP powstał już w 1991 roku [3] w wersji HTTP/0.9. Od tamtej pory doczekał się wydania wielu rozszerzeń oraz kilku nowych istotnych wersji, które wprowadzały usprawnienia w zakresie funkcjonalności (HTTP/1.0 oraz HTTP/1.1) oraz wydajności (HTTP/2.0 oraz HTTP/3).

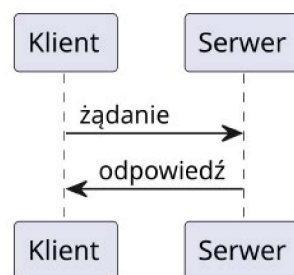
W artykule skupimy się głównie na wersji HTTP/1.1 protokołu, z odniesieniami do HTTP/2, tam gdzie będzie to potrzebne. Protokół HTTP/1.1 jest tekstowym protokołem, przez co też łatwiejszy do zrozumienia, analizy i implementacji, a mimo upływu czasu wciąż szeroko stosowany. HTTP/2, choć funkcjonalnie zbliżony do HTTP/1.1, wprowadza istotne zmiany w sposobie przesyłania danych. Mimo tych różnic podstawowe zasady działania protokołu pozostają bardzo podobne.

I ARCHITEKTURA HTTP [7]

Jak już wspomnieliśmy, HTTP opiera się na architekturze klient-serwer, gdzie to klient zawsze inicjuje komunikację. Rolą serwera jest nasłuchiwanie nadchodzących żądań (ang. *requests*) i wysyłanie odpowiednich odpowiedzi (ang. *responses*). Zarówno żądanie, jak i odpowiedź są określane jako „wiadomości” (ang. *messages*).

Często możemy spotkać inną nazwę dla klienta – *user agent*. Jest to nic innego jak program, który wysyła zapytania w imieniu użytkownika. Najbardziej popularnymi „user-agentami” są oczywiście nasze przeglądarki internetowe, chociaż może to być dowolne inne oprogramowanie, które wysyła żądania i przetwarza odpowiedź. Serwery, które mają docelowe informacje o pytanych zasobach, nazywamy serwerami backendowymi.

Najprostszy sposób komunikacji przedstawiono na Rysunku 1.



Rysunek 1. Najprostsza architektura komunikacji HTTP

Jak nietrudno się domyślić, tak prosta architektura często nie zawsze jest wystarczająca w rzeczywistych scenariuszach. Serwery obsługujące zapytania zazwyczaj nie działają w pojedynkę.

Przede wszystkim serwisy internetowe potrzebują odpowiedniej redundancji. Oznacza to, że żądania obsługiwane są przez wiele serwerów jednocześnie. Takie podejście zapewnia ciągłość działania w przypadku awarii jednego z serwerów, a także pozwala równomiernie rozłożyć obciążenie serwerów.

Aby obsłużyć taką architekturę, potrzebne są nowe elementy i w naszej architekturze pojawiają się tzw. bramki (ang. *gateways*) (inna popularna nazwa to *reverse-proxy*) i *load balancery*. Jeśli kojarzycie termin *default gateway* z poprzednich artykułów, to doprecyzujemy, że nie jest to ten sam *gateway*, o którym mówimy tutaj.

Zadaniem bramek jest właśnie odbieranie zapytań od klientów, ich wstępne przetwarzanie (jeśli jest potrzebne) oraz dystrybucja zapytań do jednego z serwerów.

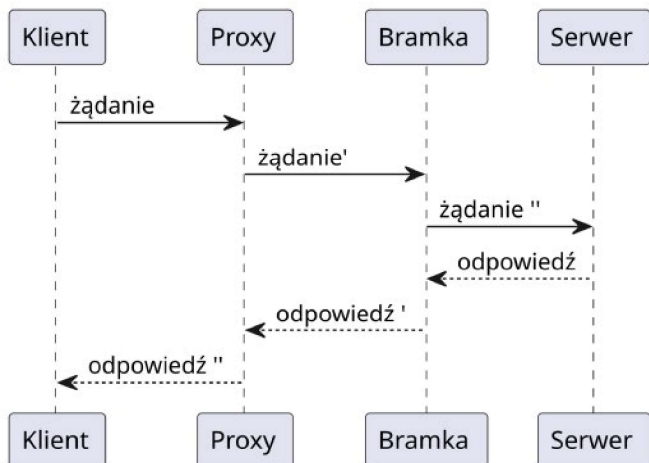
Co więcej, mogą one pełnić rolę *cache'a*. Jeśli klient pyta o zasób, który jest taki sam dla wszystkich klientów, nie ma potrzeby, żeby zapytanie trafiło bezpośrednio do serwera źródłowego i tym samym go obciążało. Dodatkowo skracają one zazwyczaj fizyczny dystans między klientem a odpowiadającym urządzeniem, co zmniejsza opóźnienie odpowiedzi.

Cache może znajdować się również po stronie klienta (np. w przeglądarkach internetowych). Nietrudno się jednak domyślić, że nie każdy zasób może być *cache'owany*. Protokół HTTP definiuje, w jaki sposób sterować zachowaniem *cache'a* (więcej na ten temat w sekcji *Cache*).

Tak samo, jak serwery mogą stać za bramkami, tak klienci mogą korzystać z tzw. *proxy*. *Proxy* może być używane z wielu powodów. Jednym z nich jest chowanie prawdziwego adresu IP (choćby istnie-

ją lepsze narzędzia do tego celu). Innym powodem jest chęć analizy naszego ruchu i modyfikowania go w locie. Jest to szczególnie przydatne w fazie implementacji serwisu. Przykładem aplikacji proxy jest aplikacja mitmproxy, której poświęciliśmy osobną sekcję artykułu.

Podsumowując, nasza architektura po uwzględnieniu wszystkich opisanych elementów może wyglądać jak ta na Rysunku 2 (może być nawet bardziej skomplikowana, ponieważ ominęliśmy niektóre elementy architektury).



Rysunek 2. Typowa architektura komunikacji HTTP

Warto wiedzieć (czego Rysunek 2 nie oddaje), że każdy z elementów może jednocześnie obsługiwać wielu klientów. Na przykład bramka może jednocześnie obsługiwać wiele zapytań od proxy oraz od klientów, a proxy może również odbierać zapytania od wielu klientów.

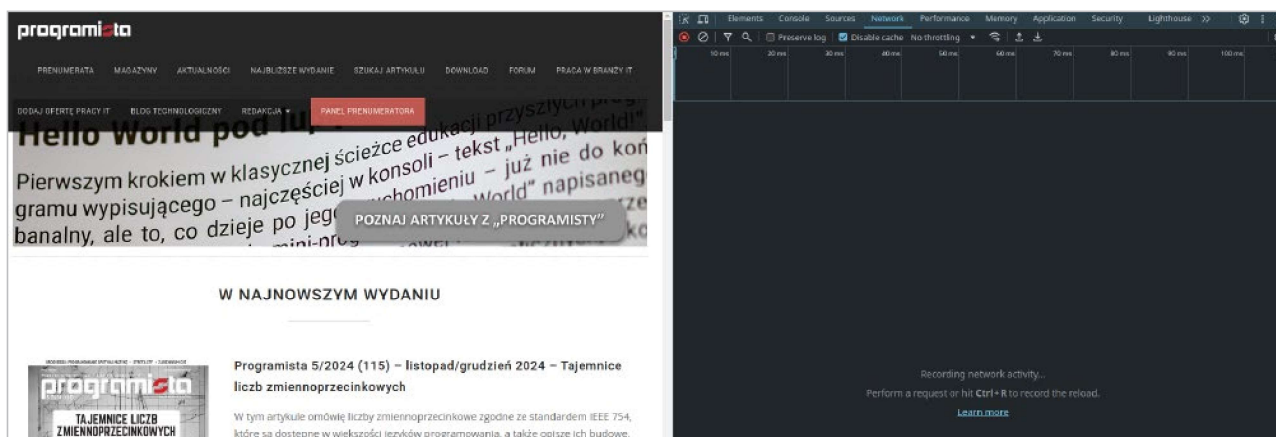
To tyle słowem wstępu. Czas zagłębić się w szczegóły zapytań HTTP.

I WIADOMOŚCI

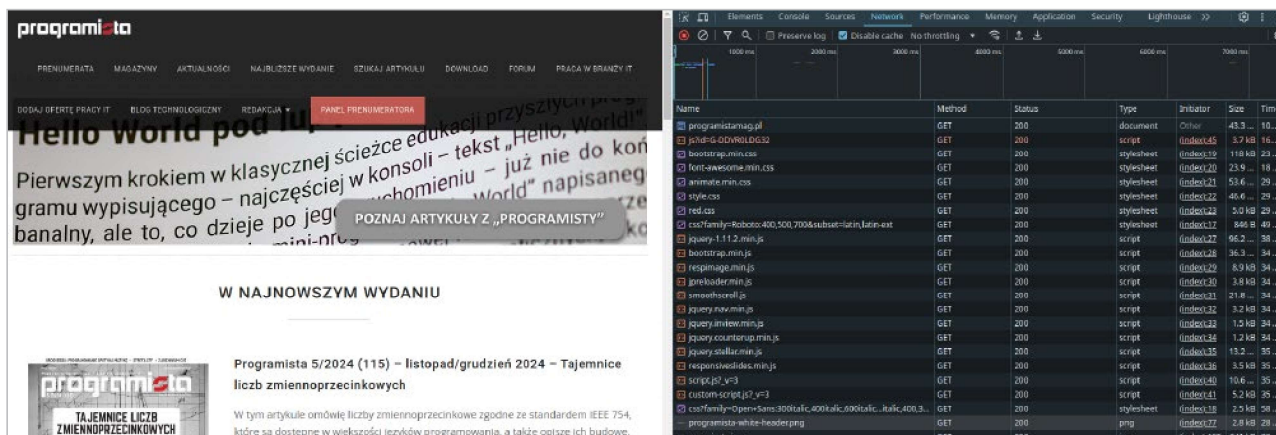
Czas włączyć naszego ulubionego user-agenta (czyli przeglądarkę) i udać się na stronę, którą wszyscy znamy: <https://programistamag.pl>, następnie otworzymy narzędzia deweloperskie (developer tools). Otworzenie ich zależy od konkretnej przeglądarki. W Chrome, Brave lub Chromium możemy to zrobić skrótem klawiszowym Ctrl+Shift+I lub klikając prawym przyciskiem myszy w dowolne miejsce na stronie i wybierając opcję Inspect. Następnie przechodzimy do zakładki Network, ponieważ to ona nas interesuje.

Nasz ekran powinien wyglądać podobnie do zrzutu ekranu na Rysunku 3.

Jak widzimy, nasza zakładka Network jest pusta. Aby zobaczyć cały ruch sieciowy związany z odwiedzaniem strony, musimy ją odświeżyć. Wynik możemy podziwiać na Rysunku 4.



Rysunek 3. Narzędzia deweloperskie w przeglądarce Brave



Rysunek 4. Zakładka Network ze wszystkimi zapytaniami związanymi z odwiedzeniem strony

Przyjrzyjmy się zatem temu, co widzimy. Kolumny, które nas najbardziej interesują, to nazwa, metoda i status. Po wybraniu danego żądania pojawi się nam więcej szczegółów rozbitych na sekcje: informacje ogólne, nagłówki odpowiedzi (ang. *response headers*) oraz nagłówki zapytania (ang. *request headers*).

W telegraficznym skrócie nazwa odnosi się do zasobu – może to być dokument HTML, obraz, styl CSS lub dowolny inny element. Każdy zasób ma swoją lokalizację, którą musimy znać, aby móc się do niego odwołać. Jest ona w określonym formacie, zwanym URI. Wkrótce przyjrzymy się temu formatowi nieco bliżej.

Wszystkie żądania na naszym rzucie ekranu mają metodę GET. Metoda mówi serwisowi, co chcemy zrobić z danym zasobem – w przypadku GET chcemy po prostu pobrać zasób. Inne popularne metody to:

- » PUT, POST – do aktualizacji zasobu, wysłania danych do przetworzenia na serwer,
- » DELETE – prośba o usunięcie zasobu.

To serwer decyduje, które metody są dostępne. Może się również zdarzyć, że wybrane metody są dostępne tylko dla wybranych użytkowników – w przeciwnym razie każdy mógłby dokonywać modyfikacji na zasobach. Co więcej, serwery mogą implementować własne metody nieopisane w standardzie HTTP.

No i w końcu statusy odpowiedzi. Z pewnością nie raz spotkaliśmy się ze stroną internetową, wyświetlającą status 404 Not Found, a może nawet 500 Internal Server Error. Statusy te służą do informowania klienta o rezultacie przetworzenia żądania. W skrócie, statusy dzielimy na 4 kategorie:

- » 200-299 – świadczą o sukcesie, z których najczęstszy to po prostu 200.
- » 300-399 – zazwyczaj mówią nam o tym, że zasób, o który pytamy, istnieje, ale został przeniesiony pod nowy adres URI.
- » 400-499 – kody błędów zapytania. Serwer nie mógł przetworzyć żądania ze względu na błąd w nim zawarty. Najpopularniejsze z nich to 404, który informuje nas o tym, że żądany zasób nie istnieje.
- » 500-599 – najbardziej problematyczne statusy. Wskazują na błąd po stronie serwera, który uniemożliwił przetworzenie żądania. Jako klienci często nie możemy nic z tym zrobić, natomiast jako programiści musimy dany problem po prostu naprawić.

Każde zapytanie, tak jak każda odpowiedź, zawiera sekcję nagłówków. To właśnie w nich przesyłane są dodatkowe informacje o zasobie. Mogą to być instrukcje dotyczące cache'owania, informacje o typie i formacie zasobu, a także o sposobie kompresji. Jeden z nagłówków to User-Agent, który identyfikuje oprogramowanie użyte do wysłania żądania – zazwyczaj przeglądarkę. Odpowiedzi HTTP mogą być w ten sposób dostosowane pod konkretną przeglądarkę, co miało znaczenie zwłaszcza podczas pierwszej wojny przeglądarkowej [4, 5], kiedy to różniły się one od siebie w niekompatybilny sposób.

I URI/URL

Zacznijmy od tego, że w praktyce terminy URI [9] i URL [8] są używane naprzemiennie. Nie jest to dziwne, ponieważ każdy URL jest

również URI (tak jak każdy kwadrat jest prostokątem), ale nie każde URI to URL. W tym artykule również będziemy używać tych nazw naprzemiennie. URI, które nie są URL, nie są używane w komunikacji HTTP.

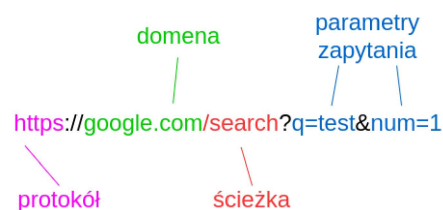
W skrócie, URL wskazuje lokalizację konkretnego zasobu w sieci. Przykład takiego URLa jest przedstawiony na Rysunku 5.



Rysunek 5. URL do obrazu przedstawiającego okładkę jednego z wydań magazynu „Programista”.

Jest to przykładowy zasób z domeny programistamag.pl. Taki URL w swojej podstawowej postaci składa się z protokołu HTTP (lub jego zaszyfrowanej postaci HTTPS), domeny identyfikującej host, na którym znajduje się zasób (oddzielonej od protokołu znakami ://), oraz ścieżki do zasobu.

To jednak nie wszystko. URL może zawierać także dodatkowe elementy, takie jak parametry zapytania (ang. *query parameters*) czy fragmenty (ang. *fragments*). Przykład z parametrami zapytania możemy zobaczyć, korzystając chociażby z wyszukiwarki Google:



Rysunek 6. URL do wyników wyszukiwania tekstu „test” w przeglądarce Google

Widzimy tutaj dodatkowe parametry zasobu:

- » q – odpowiada za tekst wyszukiwania, czyli to, co wpisujemy w pole wyszukiwarki,
- » num – ilość wyników prezentowanych na stronie.

Warto zauważyć, że pierwszy parametr jest oddzielony od ścieżki znakiem ?, natomiast kolejne parametry są rozdzielane znakiem &.

Parametry zapytania to zazwyczaj pary klucz-wartość. Nazwa parametru znajduje się po lewej stronie znaku =, a jego wartość po prawej. Warto jednak pamiętać, że jest to jedynie konwencja. Możliwy jest parametr składający się wyłącznie z nazwy (zwłaszcza popularny dla wartości logicznych) czy używanie znaku innego niż = do rozdzielania klucza od wartości.

Ostatnią nieomówioną częścią URL są tzw. fragmenty. Jak sama nazwa wskazuje – odnoszą się one jedynie do fragmentu zasobu. Żeby wskazać na część zasobu, i tak potrzebny jest cały zasób, tak więc fragmenty w przeciwieństwie do całej reszty URL nie grają roli, jeśli chodzi o komunikację klient-serwer (klienci po prostu nie wysyłają tej części URL do serwisów). Skoro fragmenty nie są wysyłane do serwera, to ich obsługa siłą rzeczy pozostaje po stronie klienta.

Co do gramatyki – fragmenty w URL pojawiają się zaraz po sekcji parametrów zapytania, oddzielone znakiem #. Przykładowy URL z fragmentem jest widoczny w Listingu 1.

Listing 1. URL do standardu URI z fragmentem wskazującym na konkretną sekcję dokumentu

`https://datatracker.ietf.org/doc/html/rfc3986#section-3.5`

Jest to dokument opisujący standard URI, a fragment wskazuje na sekcję 3.5, która opisuje nam właśnie „fragmenty”. Aplikacje zazwyczaj implementują je w ten sposób, że po wczytaniu zasobu (w tym przypadku rfc3986) przewiną widok do wskazanej sekcji.

I Kodowanie URI [10]

Jak chyba każdy format, tak i URL ma reguły dotyczące dozwolonych znaków i ich znaczenia. URL rezerwuje wiele znaków do specjalnych zastosowań (jak chociażby wspomniane już & czy ?). Znaki te mają swoje znaczenia w różnych kontekstach np. znak & oddziela od siebie parametry zapytania, a znak ? oddziela ścieżkę od parametrów. Gdybyśmy taki zarezerwowany znak chcieli użyć w znaczeniu innym niż to zarezerwowane, powinniśmy go najpierw odpowiednio zakodować.

Są również znaki, które standard wprost definiuje jako niezarezerwowane. Mogą być one używane bez obaw o niejednoznaczności w interpretacji. Co więcej (bardziej w teorii niż praktyce), możemy zamiennie używać znaków niezarezerwowanych i ich kodowanych odpowiedników.

Podczas procesu kodowania znaków ważny jest kontekst, w którym znajduje się tekst. Dla przykładu, znak &, kiedy jest używany do oddzielania kolejnych parametrów zapytania, nie musi (a nawet nie może) być kodowany (zarezerwowany znak jest używany zgodnie ze swoim przeznaczeniem). Mogłoby się jednak okazać, że potrzebujemy wykorzystać znak & w innym kontekście – w wartości parametru (lub jego nazwie). W tym przypadku znak & nie jest używany zgodnie ze swoim zarezerwowanym znaczeniem, więc musi być zakodowany.

Kodowany URL prawdopodobnie widzieliśmy wielokrotnie, jednak mogliśmy nie zdawać sobie z tego sprawy. Kodowany znak ma format %xy, gdzie xy to wartość szesnastkowa (0-255, czyli 00-ff) odpowiadająca ośmiu bitom danych. Możemy w ten sposób kodować zarówno dane binarne, jak i tekstowe.

Sprawdźmy działanie kodowania w praktyce, korzystając z naszej ulubionej wyszukiwarki google.com. Wyszukajmy frazę ampersand& – zostaniemy przeniesieni na stronę o URL: `https://www.google.com/search?q=ampersand%26...`. Widzimy, że ampersand został zamieniony na kodowaną wersję - %26, a to właśnie dlatego, że zarezerwowany znak został użyty jako wartość parametru zapytania w innym znaczeniu niż to zarezerwowane. W tym miejscu chciałbym jeszcze dodać jedną rzecz – niektóre implementacje (a w zasadzie spora ich liczba) zamiast kodować znak spacji do %20, zamienia go na znak +. O ile w tym przykładzie tego nie zobaczyliśmy, to w praktyce można spotkać się z takim zachowaniem dość często.

Po stronie serwisu należy pamiętać o tym, aby przetworzyć URI w celu rozdzielania na poszczególne składowe (ścieżka, lista parametrów, adres/domena itp.) przed tym, jak dokonamy dekodowania zna-

ków. W przeciwnym razie podczas dekodowania URI może dojść do zmiany jego znaczenia. Przykładem takiego problematycznego URI będzie wymyślony adres `http://popularne-seriale.pl/jak%20to%20jest%20zrobione%3F/s1e1?start=150`. Znaki %20 to zakodowane spacje, natomiast %3F to zakodowany znak ?. Po jego zdekodowaniu otrzymamy: `.../jak to jest zrobione?/s1e1?start=150`. Jak widać, w URI mamy teraz 2 znaki ?, co spowoduje, że nasze parametry zapytania zostaną zinterpretowane jako klucz /s1e1?start o wartości 150 zamiast start o wartości 150. Przez taki błąd ścieżka została błędnie zinterpretowana.

Listę wszystkich znaków zarezerwowanych, jak i niezarezerwowanych znajdziemy oczywiście w specyfikacji. Warto jednak skorzystać z funkcji dostarczanych we frameworkach lub bibliotekach standardowych języków. W JavaScript do kodowania URI można użyć funkcji `encodeURIComponent` oraz `encodeURIComponent`. Jak wspomnieliśmy, kodowanie jest zależne od kontekstu, a więc funkcje te powinny być użyte odpowiednio do miejsca, w którym dany tekst występuje. I tak np. `encodeURIComponent` powinniśmy używać, kiedy chcemy kodować tekst w parametrze zapytania, a `encodeURIComponent` w pozostałych przypadkach.

Skoro znamy już URI, przejdźmy do bardziej szczegółowego opisu wiadomości HTTP. Kodowanie oraz URI (a zazwyczaj jego ścieżka) będą częścią zapytania wysyłanego do serwera.

I Żądania i odpowiedzi

Komunikacja HTTP jest oparta na żądaniach (czasami nazywanych również zapytaniami) i odpowiedziach na nie. Chociaż wspomnieliśmy wcześniej, że narzędzia deweloperskie w przeglądarce są bardzo przydatne, to jako programiści serwisów internetowych zazwyczaj będziemy korzystać z innych narzędzi.

Wprowadźmy więc kilka nowych zabawek do naszego arsenału.

I cURL i Wireshark

Zacznijmy od narzędzia Wireshark do analizy zapytań HTTP. Niestety (albo stety, jeśli chodzi o prywatność danych) większość serwisów komunikuje się, wyłącznie używając zaszyfrowanych danych. Znaczy to tyle, że w aplikacji Wireshark nie ujrzymy „surowej” komunikacji HTTP (tak naprawdę da się to zrobić, podając odpowiednie klucze szyfrujące, ich uzyskanie często jest dość problematyczne i łatwiej skorzystać z później omawianego proxy). Z tego względu w tym artykule Wireshark nie będzie naszym głównym narzędziem do analizy komunikacji (prześledzimy jedynie najprostsze przypadki).

Aby wymusić brak szyfrowania danych, użyjemy narzędzia `curl` do wysyłania żądań, które w najprostszej formie wygląda następująco:

Listing 2. Najprostsze użycie narzędzia curl

```
curl http://example.com
```

Argumentem programu jest URL naszego zasobu. Ponieważ podajemy protokół HTTP (a nie HTTPS), nasze narzędzie spróbuje nawiązać połączenie nieszyfrowane. Po wysłaniu żądania dostajemy w odpowiedzi dokument HTML. Całość możemy zaobserwować w Wiresharku, co przedstawiono na Rysunku 7.

No.	Time	Source	Destination	Protocol	Length	Info
436	32.98961...	10.11.0.25	93.184.215.14	TCP	74	56894 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2696185520 TSecr=0 WS=128
439	33.08856...	93.184.215.14	10.11.0.25	TCP	74	80 → 56894 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=283407507 TSecr=26
440	33.08865...	10.11.0.25	93.184.215.14	TCP	66	56894 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2696185619 TSecr=283407507
441	33.08889...	10.11.0.25	93.184.215.14	HTTP	141	GET / HTTP/1.1
442	33.29096...	93.184.215.14	10.11.0.25	TCP	66	80 → 56894 [ACK] Seq=1 Ack=76 Win=65536 Len=0 TSval=283407610 TSecr=2696185619
443	33.29096...	93.184.215.14	10.11.0.25	HTTP	1663	HTTP/1.1 200 OK (text/html)
444	33.29103...	10.11.0.25	93.184.215.14	TCP	66	56894 → 80 [ACK] Seq=76 Ack=1598 Win=62720 Len=0 TSval=2696185821 TSecr=283407611
445	33.29135...	10.11.0.25	93.184.215.14	TCP	66	56894 → 80 [FIN, ACK] Seq=76 Ack=1598 Win=62720 Len=0 TSval=2696185822 TSecr=283407611
446	33.39185...	93.184.215.14	10.11.0.25	TCP	66	80 → 56894 [FIN, ACK] Seq=1598 Ack=77 Win=65536 Len=0 TSval=283407811 TSecr=2696185822
447	33.39194...	10.11.0.25	93.184.215.14	TCP	66	56894 → 80 [ACK] Seq=77 Ack=1599 Win=62720 Len=0 TSval=2696185922 TSecr=283407811

Rysunek 7. Zrzut ruchu sieciowego spowodowanego aplikacją cURL

Na powyższym rysunku widzimy komunikację dwóch protokołów TCP oraz HTTP. Ponieważ HTTP korzysta z protokołu TCP, na rysunku zobaczymy komunikację TCP, zaczynającą się od charakterystycznej sekwencji SYN → SYN, ACK → ACK świadczącej o nawiązaniu połączenia TCP. Kolejno widzimy żądanie GET / HTTP/1.1, a następnie odpowiedź HTTP/1.1 200 OK. Na koniec następuje zakończenie połączenia TCP (FIN, ACK → FIN, ACK → ACK) na protokole TCP.

■ Zapytanie i jego nagłówki

Przjrzyjmy się bliżej formatowi zapytania. Klikając prawym przyciskiem myszy na zapytanie GET, przejdźmy do opcji Follow → HTTP Stream. Naszym oczom pojawi się zarówno zapytanie, jak i odpowiedź.

Listing 3. Żądanie HTTP z dodanymi explicite znakami nowych linii

```
GET / HTTP/1.1\r\n
Host: example.com\r\n
User-Agent: curl/8.10.1\r\n
Accept: */*\r\n
\r\n
```

W Listingu 3 dodałem odpowiednio znaki nowych linii. W formacie HTTP sekwencja znaków nowej linii to zawsze \r\n niezależnie od używanego systemu operacyjnego.

Pierwsza linia zapytania (ang. *request line*), czyli GET / HTTP/1.1, informuje nas o metodzie zapytania (GET), celu (ang. *target*), który jest ścieżką zasobu, oraz wersji protokołu HTTP. Warto zwrócić uwagę, że w URI ścieżka może być pusta, natomiast format zapytania HTTP wymaga podanie celu jako / dla pustej ścieżki.

Po pierwszej linii następuje seria nagłówków, które są parami klucz-wartość. Nazwa nagłówka znajduje się po lewej stronie znaku :, a jego wartość po prawej. Koniec sekcji nagłówków rozpoznajemy po dwóch następujących po sobie znakach nowej linii CRLF (Carriage Return, Line Feed), czyli \r\n\r\n.

W dalszej części zapytania znajdziemy nagłówki, które spotkamy praktycznie w każdym żądaniu:

- » **Host** – nazwa hosta – w większości przypadków będzie to nazwa domenowa, którą podajemy w URI. Rzadziej (głównie podczas developmentu i testowania) spotkamy tam adres IP czy nazwę localhost. Można by się zastanawiać, dlaczego wysyłamy do serwisu jego domenę – nie ma przeszkód, żeby to serwis wiedział, jaką domenę obsługuje. Faktycznie istnieje wiele serwisów, gdzie podanie tego nagłówka jest opcjonalne, jednak często się zdarza, że serwer obsługuje ruch dla wielu domen. W takim

przypadku odpowiednie reguły przekierowywania ruchu będą bazowały właśnie na tym nagłówku.

- » **User-Agent** – informuje serwis o tym, jakie oprogramowanie wysłało zapytanie. Może to być użyte, gdyby serwer chciał dostosować odpowiedź pod konkretnego agenta.
- » **Accept** [11] – ten nagłówek o magicznej wartości informuje serwer o formatach danych, które klient jest w stanie obsłużyć. Na przykład serwer przechowuje dokumenty, które są dostępne jako zwykłe pliki tekstowe, jak i pliki PDF. Klient może poinformować, który z formatu danych preferuje. Jest to jeden z nagłówków biorących udział w procesie tzw. negocjacji kontentu (ang. *content negotiation*). W ramach tej negocjacji możemy informować serwer o preferowanych językach czy kompresji danych. Więcej o tym można poczytać w bibliografii – Content Negotiation [12].

W ogólnym formacie żądania może wystąpić tzw. ciało (nie występuje ono w przypadku metody GET). Ciało wysyłamy, kiedy chcemy serwerowi wysłać dane do przetworzenia lub zasób do przechowania. Bardzo często na stronach internetowych spotykamy się z różnymi formularzami (np. podczas zakupów) – takie formularze są często wysyłane do serwera z metodą POST oraz właśnie ciałem zawierającym wypełniony formularz.

■ Odpowiedź i jej nagłówki

Odpowiedź ma bardzo podobną strukturę i wygląda następująco:

Listing 4. Odpowiedź HTTP na żądanie z Listingu 3

```
HTTP/1.1 200 OK\r\n
Accept-Ranges: bytes\r\n
Age: 518869\r\n
Cache-Control: max-age=604800\r\n
Content-Type: text/html; charset=UTF-8\r\n
Date: Wed, 27 Nov 2024 18:52:37 GMT\r\n
Etag: "3147526947"\r\n
Expires: Wed, 04 Dec 2024 18:52:37 GMT\r\n
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT\r\n
Server: ECAcc (nyd/D11B)\r\n
Vary: Accept-Encoding\r\n
X-Cache: HIT\r\n
Content-Length: 1256\r\n
\r\n
<!doctype html>
<html>
<head>
...
```

Podobnie jak w przypadku zapytania, odpowiedź ma specjalny format pierwszej linii, którą nazywamy linią statusu (ang. *status line*). W tej linii serwer wysła informacje o powodzeniu (lub niepowodzeniu) przetworzenia żądania. Tak samo jak w żądaniu poszczególne

nagłówki oddzielone są sekwencją nowej linii (`\r\n`). W odpowiedzi na żądanie zostało wysłane także ciało (oddzielone od nagłówków pustą linią, czyli sekwencją `\r\n\r\n`) zawierające żądany zasób.

Co do nagłówków, to widzimy ich znacznie więcej niż w zapytaniu (a nawet więcej niż jest to naprawdę potrzebne). Większość z nich jest związana z cache'm oraz tzw. świeżością (ang. *freshness*) zasobu, o czym opowiemy szerzej w sekcji Cache. Do najważniejszych nagłówków związanych z cache'm należą:

- » Cache-Control
- » Etag
- » Expires
- » Last-Modified
- » Vary

A więc jak widać większość nagłówków w tym przykładzie odpowiada za kontrolę mechanizmu cache'owania zasobów.

■ Ciało wiadomości

Szczególnie ważny nagłówek to Content-Length, który informuje o rozmiarze ciała w zapytaniach i odpowiedziach. Dzięki temu odbiorca wiadomości wie, gdzie znajduje się jej koniec (w wiadomościach bez ciała o końcu wiadomości świadczy sekwencja `\r\n\r\n`).

Innym nagłówkiem, który może informować odbiorcę o tym, gdzie znajduje się koniec wiadomości, jest nagłówek Transfer-Encoding z wartością chunked, używany wtedy, gdy nie znamy z góry rozmiaru wiadomości (ta może być generowana dynamicznie, w locie).

W takim przypadku ciało jest ułożone z sekwencji:

- » rozmiar części wiadomości w bajtach (zapisanych szesnastkowo bez zwyczajowego prefiksu `0x`),
- » CRLF, czyli sekwencja `\r\n`,
- » część wiadomości.

Taka sekwencja się powtarza, aż wysłana nie zostanie cała wiadomość, a na końcu zostanie wysłana specjalna część wiadomości o rozmiarze 0, czyli `0\r\n\r\n`. Ponieważ jak już wspomnieliśmy HTTP jest protokołem tekstowym, rozmiar w bajtach jest podany tekstowo (tak samo jak nagłówek Content-Length, co możemy zobaczyć na przykładzie prostego serwera hello world).

Na Rysunku 8 widzimy część składni odpowiedzi, która wskazuje, że kolejna część odpowiedzi będzie mieć 5 bajtów. Składa się ona z 3 bajtów – `0x35`, `0x0d`, `0x0a`, co tekstowo odpowiada znakom 5, `\r`, `\n`.

■ Rola metod w zapytaniach

HTTP definiuje tylko kilka metod [14]. Są to:

- » GET
- » HEAD
- » POST
- » PUT
- » OPTIONS
- » DELETE
- » CONNECT
- » TRACE

Poza nimi inne standardy mogą definiować więcej metod, a także same aplikacje mogą definiować nowe metody.

Dwie z wymienionych metod nie są w naszym zainteresowaniu. Jedną z nich to TRACE, która wykorzystywana jest do testowania aplikacji. Serwisy na zapytanie o metodę TRACE powinny w ramach odpowiedzi odesłać otrzymane zapytanie. Z powodów bezpieczeństwa metody tej z reguły się nie implementuje (zazwyczaj w ogóle).

Kolejną metodą, której nie będziemy poświęcać czasu, to CONNECT, która jest przeznaczona do komunikacji z serwerami proxy. Metoda ta informuje serwis proxy, z jakim serwerem należy się połączyć. Po takim żądaniu wszystkie dane wysłane do serwera proxy będą przekazane do docelowego serwera.

Dopóki nie będziemy implementować połączeń z serwerami proxy, ani samych serwisów proxy, nie musimy o tym pamiętać. Większość bibliotek i frameworków HTTP obsługuje już połączenia z proxy, dzięki czemu my nie musimy się już tym martwić.

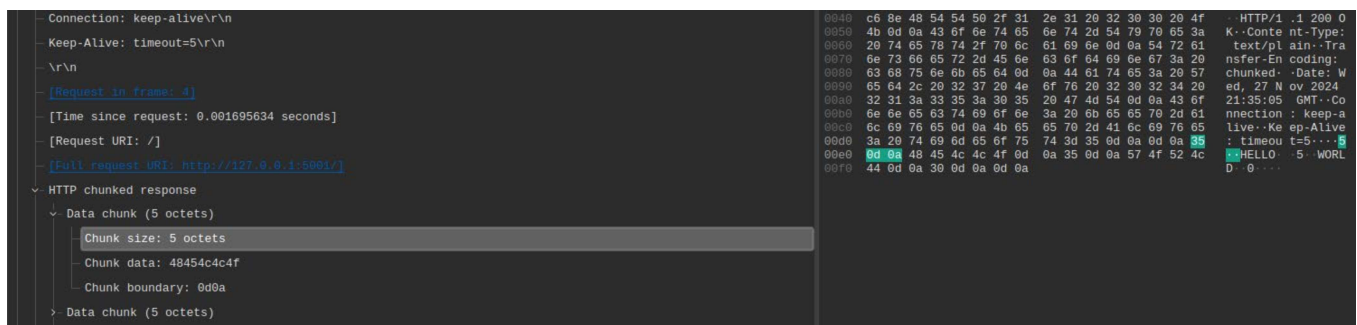
■ GET, HEAD

Najczęściej używaną i najbardziej podstawową metodą (dostępną od pierwszej wersji HTTP) jest GET, która służy do pobierania zasobu [15]. Żądania GET nie mają ciała, a ewentualne argumenty możemy przekazywać w ramach parametrów zapytania lub jako nagłówki.

Przy zapytaniu z użyciem aplikacji curl metoda GET jest używana domyślnie. Możemy jednak jawnie przekazać metodę przez parametr `-X` lub `--request`, np.:

Listing 5. Przykładowe żądanie GET z użyciem aplikacji curl

```
curl -X GET https://www.google.com
```



Rysunek 8. Przykład pofragmentowanej odpowiedzi widzianej w aplikacji Wireshark

W niektórych mechanizmach związanych z bezpieczeństwem (jak np. CORS, o którym można poczytać w bibliografii [29]) przyjmowane jest, że metoda GET nie zmienia stanu serwera. Warto przestrzegać tego założenia we własnych implementacjach serwisów, aby uniknąć niezgodności.

Podobną do GET jest metoda HEAD [16]. Odpowiedź na zapytanie będzie prawie taka sama, jak w przypadku GET, z tą różnicą, że nie zawiera ciała. Metoda ta jest przydatna, jeśli chcemy zaoszczędzić ilość przesyłanych danych, a potrzebujemy jedynie metadane (nagłówki) odpowiedzi. W nagłówkach odpowiedzi możemy znaleźć takie informacje jak rozmiar zasobu (Content-Length) czy datę jego ostatniej modyfikacji (nagłówek Last-Modified).

Podobnie i w tym przypadku istnieje założenie, że stan serwisu po obsłużeniu zapytania metodą HEAD nie zmieni się.

Wspomnieliśmy, że metodę zapytania w aplikacji curl możemy zmieniać poprzez parametr `-X` lub `--request`, jednak w przypadku metody HEAD występuje wyjątek. Standardowy sposób rozpoznawania długości odpowiedzi bazuje na nagłówku Content-Length lub Transfer-Encoding. W przypadku metody HEAD te nagłówki zostaną wysłane, dokładnie tak samo jak przy metodzie GET, jednak ciało nie będzie wysłane. Jeśli więc użyjemy flagi `-X` lub `--request` z wartością HEAD, aplikacja curl będzie w nieskończoność (aż do przekroczenia dość długiego czasu oczekiwania) czekać na ciało wysłane z serwera. Dopiero parametr `-I` lub `--head` sprawi, że aplikacja nie będzie czekać na ciało, a dodatkowo wyświetli nam w przyjazny sposób nagłówki odpowiedzi.

Na marginesie warto wspomnieć o pewnej nakładce na aplikację curl – curlie, która swój interfejs opiera na jeszcze innej aplikacji httpie, a pod spodem używa aplikacji curl.

Przykładowe zapytanie HEAD będzie więc wyglądać następująco: `curl -I http://example.com` z użyciem aplikacji curl lub `curlie HEAD http://example.com` z użyciem aplikacji curlie.

Odpowiedź zapytania została przedstawiona w Listingu 6

Listing 6. Odpowiedź z serwera na żądanie HEAD

```
HTTP/1.1 200 OK
Content-Encoding: gzip
Accept-Ranges: bytes
Age: 477947
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Thu, 28 Nov 2024 19:08:56 GMT
Etag: "3147526947+gzip"
Expires: Thu, 05 Dec 2024 19:08:56 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECAcc (nyd/D12C)
X-Cache: HIT
Content-Length: 648
```

Tak jak wspomnieliśmy – widzimy nagłówek Content-Length, jednak nie otrzymujemy żadnego ciała związanego z zapytaniem.

PUT, POST

Metody PUT i POST to dwie chyba najczęściej mylone metody, ponieważ obie modyfikują stan serwisu. Robią to jednak na różny sposób, dlatego przyjrzymy im się nieco bliżej. Za ich pomocą możemy zarówno utworzyć nowe zasoby, modyfikować je, jak również zlecić serwisowi przetwarzanie danych [17], [18]. Ostatecznie wszystkie opisane właściwości tych metod (nie tylko PUT i POST) to jedynie zalecenia, a zachowanie serwisu zależy tylko od jego implementacji.

Metody te różnią się pewną cechą żądań, zwaną *idempotentnością* [19].

Idempotentność polega na tym, że niezależnie od liczby wysłanych identycznych żądań, stan serwisu po wysłaniu ostatniego żądania będzie taki sam jak po wysłaniu pierwszego żądania. I tak metoda PUT jest idempotentna, ale POST już nie.

Bardziej przyjazne wyjaśnienie może być następujące. Metodą POST wysyłamy dane do serwisu, aby je *przetworzyć*. Może to być np. złożenie zamówienia przy zakupach online czy zlecenie przelewu bankowego. Wysłanie dziesięciu takich żądań skutkuje odpowiednio utworzeniem 10 zamówień i dziesięcioma przelewami w banku.

Metodą PUT natomiast wysyłamy dane do serwisu, aby je *przechować*. Możemy np. w ten sposób wysłać nowy plik (lub nadpisać istniejący) chociażby do serwisu One Drive [20].

Przykład żądania PUT:

Listing 7. Szablon żądania do serwisu One Drive w celu utworzenia nowego pliku lub modyfikacji istniejącego

```
PUT /me/drive/root:/FolderA/FileB.txt:/content
Content-Type: text/plain

Treść pliku jest ciałem zapytania
```

Idempotentność takiego żądania polega na tym, że nawet jeśli wyślemy trzy takie żądania jedno po drugim, to w serwisie i tak powstanie jeden plik ze ścieżką `root:/FolderA/FileB.txt` o zawartości Treść pliku jest ciałem zapytania.

Warto wspomnieć jeszcze o metodzie PATCH. Jest ona podobna do metody PUT, jednakże służy do aktualizacji tylko części istniejącego zasobu (pozwala zaoszczędzić transfer danych, a przez to często przyspieszyć całą operację).

Spójrzmy teraz na przykłady metody POST. Są one często używane przy przetwarzaniu formularzy na stronach internetowych (jeśli kiedykolwiek ktoś tworzył treści w HTML, zapewne kojarzy znacznik HTML – form).

Demo takiego formularza możemy znaleźć chociażby na stronie https://tryphp.w3schools.com/showphp.php?filename=demo_form_post. Prześledźmy, jak działa obsługa tego formularza, patrząc na zakładkę Network w narzędziach deweloperskich przeglądarki.

Po przejściu na stronę zobaczymy następujący formularz, który ma pole Name oraz E-mail. Kod źródłowy HTML strony wygląda następująco:

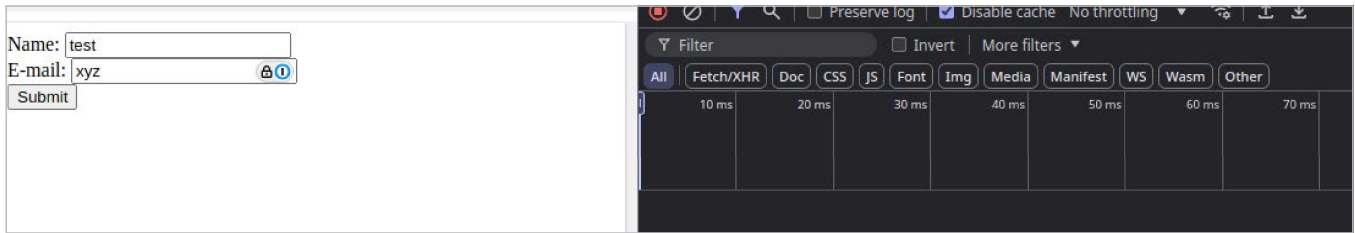
Listing 8. Kod HTML formularza

```
<!DOCTYPE HTML>
<html>
<body>

<form action="welcome.php" method="post">
Name: <input type="text" name="name"><br>
E-mail: <input type="text" name="email"><br>
<input type="submit">
</form>

</body>
</html>
```

Jak widzimy, formularz wprost definiuje, że użytą metodą będzie POST. Definiuje on także parametry: `name` oraz `email`. Przed kliknięciem przycisku submit strona prezentuje się następująco:



Rysunek 9. Prosty formularz razem z pustą zakładką ruchu sieciowego

Zaraz po kliknięciu w przycisk submit będziemy mogli zaobserwować żądanie POST. Ważne jest również to, w jaki sposób dane zostały wysłane przez tę metodę. Nietrudno się domyślić, że skoro nie są one częścią URI, to zostały umieszczone w ciele żądania, co widzimy na Rysunku 10 oraz 11.

Ciało takiego żądania to: `name=test&email=xyz` (Rysunek 11). Widzimy, że ciało żądania zawiera wszystkie parametry formularza przesłane w formacie podobnym do parametrów zapytania w URI, gdzie kluczem jest pole formularza, a jego wartość jest podana po prawej stronie znaku `=`.

Bazując na tym przykładzie, nietrudno sobie wyobrazić, że formularz dotyczy np. kupna produktu ze strony internetowej. W takim przypadku formularz mógłby mieć następujące pola:

- » imię i nazwisko
- » adres dostawy
- » lista produktów

Które mogłyby być wysłane właśnie żądaniem POST. Brak idempotentności takiego żądania oznacza, że kolejne żądania powodują kolejnymi zamówieniami złożonymi na te same produkty i z dostawą na ten sam adres.

Warto wspomnieć, że istnieją w użyciu również inne formaty danych wysyłane przez formularze. Jednym z nich jest multipart/form-

-data. Jest to format przystosowany do wysyłania zawartości plików. W jednym żądaniu/formularzu możemy zawrzeć ich kilka. W takim formacie kolejne pliki oddzielone są wygenerowanym przez klienta unikatowym tekstem (tzw. boundary), którego nie znajdziemy w zawartości żadnego z wysyłanych plików, a który je rozdziela. Tekst ten przesyłany jest w nagłówku żądania. Po każdym wystąpieniu boundary następuje sekwencja dodatkowych nagłówków dotyczących konkretnego pliku (np. jego nazwa), a po kończącej nagłówki sekwencji `\r\n\r\n` załączony jest sam plik [30].

DELETE

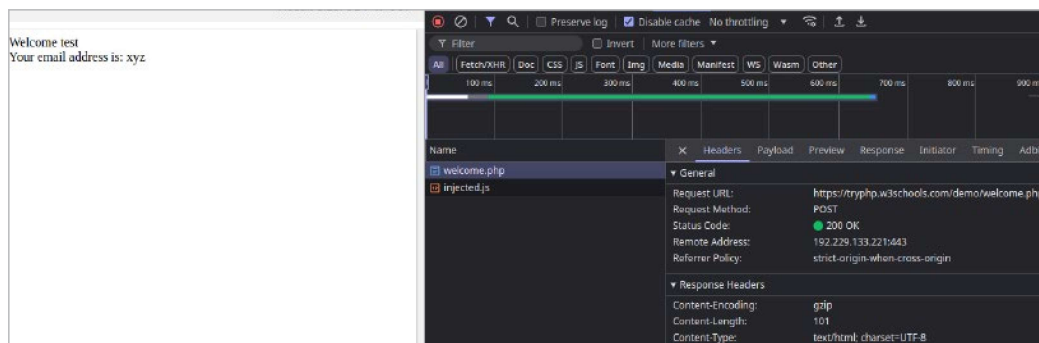
Metoda DELETE informuje o chęci usunięcia zasobu z serwisu. Żądanie takie jest idempotentne jak metoda PUT, czyli zarówno jedno, jak i wiele żądań wysłanych po sobie będzie miało ten sam skutek – zasobu w serwisie nie będzie.

Podobnie możemy się odwołać tutaj do serwisu One Drive i sposobu na usuwanie plików z serwisu [21].

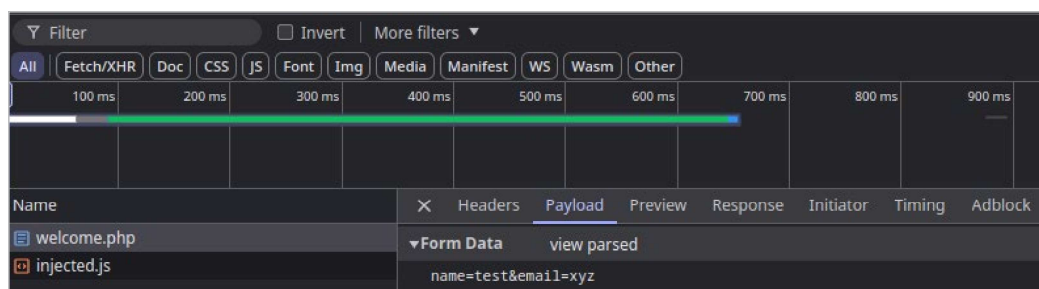
Przykładowe żądanie o usunięcie zasobu wygląda następująco:

Listing. 9 Szablon żądania o usunięcie pliku z serwisu OneDrive

```
DELETE /me/drive/items/{item-id}
```



Rysunek 10. Zakładka sieciowa zaraz po wysłaniu żądania post poprzez przycisk formularza



Rysunek 11. Ciało żądania POST wysłanego po naciśnięciu przycisku Submit

W tym przypadku {item-id} otrzymujemy chociażby w odpowiedzi na żądanie utworzenia pliku.

OPTIONS

Metoda OPTIONS jest dość nietypowa i w praktyce rzadko będziemy wysyłać takie zapytania. Nie służy ona do manipulowania w żaden sposób zasobami serwisu, a jest pomocna jedynie do uzyskania informacji o tym, jakie metody wspiera dany zasób [22]. W odpowiedzi na takie zapytanie zobaczymy nagłówek Allow, który zawiera listę wspieranych metod oddzielonych przecinkami.

Najczęstsze żądania OPTIONS dotyczą jednak mechanizmu „preflight CORS request”, do poczytania o którym zachęcam we własnym zakresie [29].

Kody statusu w odpowiedziach

Dla kompletności warto spojrzeć jeszcze raz na kody statusów. Jak już mówiliśmy, pierwsza linia żądania składa się z metody, ścieżki do zasobu oraz wersji HTTP. Natomiast pierwsza linia odpowiedzi składa się z wersji HTTP (w następnej sekcji opowiemy, dlaczego jest ona potrzebna), kodu statusu i tzw. powodu (ang. *reason*). W większości przypadków powód odpowiada 1:1 kodowi statusu (każdy kod ma swój domyślny powód, który jest używany w znakomitej większości przypadków).

Kody statusu wpadają w jedną z kilku kategorii [24]:

- » Odpowiedzi informacyjne 100-199
- » Odpowiedzi świadczące o prawidłowym przetworzeniu zapytania 200-299
- » Odpowiedzi przekierowujące 300-399
- » Błędy klienckie 400-499
- » Błędy serwera 500-599

W praktyce rzadko spotkamy się z odpowiedziami z kategorii informacyjnych, dlatego też nie będziemy ich tutaj omawiać.

Najczęściej jednak będziemy się spotykać z odpowiedziami 2xx, które świadczą o poprawnym przetworzeniu żądania. Z tej rodziny najczęściej możemy spotkać kody z Tabeli 1.

Kod statusu	Powód (reason)	Co znaczy?
200	OK	Żądanie zostało prawidłowo obsłużone. Często ten kod statusu jest używany w odpowiedzi na żądanie GET, w którym to przypadku ciało odpowiedzi będzie zawierać żądany zasób
201	Created	Żądanie zostało prawidłowo obsłużone, co poskutkowało utworzeniem nowego zasobu w serwisie. Zazwyczaj odpowiedź zawiera informację o lokalizacji nowego zasobu (zwłaszcza jeśli powstał on w wyniku żądania POST)
202	Accepted	Dość specyficzny kod statusu, który oznacza, że żądanie jest prawidłowe, ale nie zostało (jeszcze) przetworzone. Przeważnie kod ten jest używany w asynchronicznych API, gdzie przetwarzanie żądań trwa długo, a wynik przetwarzania możemy uzyskać, wysyłając dodatkowe żądania do serwisu. Zazwyczaj odpowiedź 202 będzie zawierała informację o tym, w jaki sposób uzyskać więcej informacji o przetwarzanym żądaniu

Kod statusu	Powód (reason)	Co znaczy?
204	No Content	Znaczy to samo, co zapytanie GET (żądanie zostało prawidłowo obsłużone), jednak ciało odpowiedzi nie zawiera żadnych informacji. Zazwyczaj wysyłane na żądania PUT w związku z aktualizacją istniejących treści

Tabela 1. Najczęstsze kody statusów z rodziny 2xx

Kolejną rodziną statusów są kody 3xx. Są to statusy „pomocnicze”. Zazwyczaj informują nas, że zasób, o który prosimy, znajduje się obecnie pod nowym adresem lub informuje o świeżości zasobu (o czym więcej w sekcji Cache).

Kod statusu	Powód (reason)	Co znaczy?
301	Moved Permanently	Oznacza, że zasób kiedyś był dostępny pod danym URLem, ale obecnie znajduje się w nowym miejscu, które wskazuje nagłówek Location w odpowiedzi na żądanie. Przeglądarki lub inne aplikacje działające w imieniu klienta (user agents) zazwyczaj automatycznie wysyłają kolejne, niezmienione zapytanie pod nowy URL
302	Found	Podobnie jak 301, jednak z tą różnicą, że zasób zmienił lokalizację jedynie tymczasowo
304	Not Modified	Ten kod statusu jest dość powszechny, zwłaszcza jeśli mamy do czynienia z cache’em. Przeglądarki, lub inni aktorzy posiadający cache, mogą pytać o to, czy dany zasób zmienił się na serwerze, wysyłając do niego specjalny identyfikator (tzw. <i>etag</i>) lub datę ostatniej modyfikacji. Jeśli zasób nie zmienił się, serwis odpowiada właśnie kodem 304, a w przeciwnym razie kodem 2xx z właściwymi headerami oraz ciałem odpowiedzi. Więcej na ten temat w sekcji Cache

Tabela 2. Najczęstsze kody statusów z rodziny 3xx

Kolejną rodziną odpowiedzi są wartości 4xx. Świadczą one o błędzie podczas przetwarzania zapytania, który spowodowany jest błędem użytkownika. Zazwyczaj będziemy mieli do czynienia ze złym formatem wysłanego żądania lub nieistniejącymi zasobami.

Kod statusu	Powód (reason)	Co znaczy?
400	Bad Request	W wysłanym żądaniu znajduje się błąd. Problem może być w składni zapytania lub jego semantyce (w zależności od serwisu i API różne rzeczy mogą oznaczać problem żądania). Wysłanie ponownie żądania bez jego modyfikacji zazwyczaj skutkuje taką samą odpowiedzią
404	Not Found	Żądany zasób nie istnieje. Zazwyczaj jest to odpowiedź na zapytanie GET lub PUT o dany zasób. Istnieje również możliwość, że serwis odpowiada tym kodem błędu, kiedy nie mamy dostępu do danego zasobu (aby schować informację o istnieniu zasobu przed użytkownikami nieuprawnionymi do jego odwiedzenia)
409	Conflict	Konflikt możemy zaobserwować w sytuacji, gdy 2 lub więcej żądań o modyfikację zasobu będzie w tym samym czasie przetwarzane przez serwis.

Tabela 3. Najczęstsze kody statusów z rodziny 4xx

Jak widać, kodów jest całkiem sporo, a to jedynie najczęstsze z nich. Nie należy się ich jednak uczyć na pamięć, zawsze możemy znaleźć odpowiednie np. na stronie developer.mozilla.com.

I Po co wersja HTTP w odpowiedzi?

Odpowiedź HTTP zawiera w pierwszej linii informację o wersji protokołu HTTP, który został użyty. Nie chodzi jednak o to, by wersja protokołu w odpowiedzi zawsze była taka sama, jak ta w żądaniu. Klient i serwer mogą obsługiwać różne wersje HTTP, a wersja w odpowiedzi wskazuje, którą z nich serwer obsługuje w danym momencie. Jeśli mowa o HTTP/1.1, to dziś raczej ciężko znaleźć serwis, który by nie był z tą wersją kompatybilny.

Niemniej jednak w poprzednich wersjach HTTP niektóre cechy protokołu się różniły. I tak np. domyślnie w HTTP/1.1 po uzyskaniu odpowiedzi klient i serwer powinny utrzymywać połączenie między sobą, a w HTTP/1.0 nie. Aby nie dochodziło do nieporozumień, serwer odpowiadając, zawiera najwyższą wersję HTTP, którą wspiera (nie większą niż ta w żądaniu).

Co ciekawe, pierwsza wersja HTTP – 0.9 – nie zawierała w ogóle informacji o wersji HTTP. Pierwsza linia zapytania składała się jedynie z metody (zawsze GET) oraz ścieżki, a nagłówki były pomijane.

Obecnie aplikacja cURL (a przynajmniej wersja zainstalowana na moim środowisku) nie wspiera wysyłania żądań w standardzie 0.9, ale żądanie takie możemy skonstruować ręcznie np. poprzez aplikację nc:

Listing 10. Przykład użycia aplikacji nc, aby skonstruować własne żądanie

```
$ nc example.com 80
> GET /
>
>
```

Zwróćmy uwagę na liczbę 80. Warto wspomnieć, że protokół TCP zawiera w każdym pakiecie informację o porcie. Port jest w zasadzie tylko liczbą, ale pełni ważną rolę, a mianowicie używany jest do dopasowania pakietu do konkretnej aplikacji działającej na systemie (pod danym adresem IP, do którego jest adresowany pakiet).

Zwyczajowo nieszyfrowana komunikacja HTTP odbywa się na porcie 80, a szyfrowana HTTPS na porcie 443.

W odpowiedzi otrzymamy:

Listing 11. Odpowiedź na żądanie z Listingu 10

```
HTTP/1.0 404 Not Found
Content-Type: text/html
Date: Thu, 05 Dec 2024 22:00:02 GMT
Server: ECAcc (dcd/7D26)
Content-Length: 345
Connection: close

<?xml version="1.0" encoding="iso-8859-1"?>
<!DOCTYPE html PUBLIC
"-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/
xhtml1-transitional.dtd">
<html
xmlns="http://www.w3.org/1999/xhtml"
xml:lang="en"
lang="en">
  <head>
    <title>404 - Not Found</title>
  </head>
  <body>
    <h1>404 - Not Found</h1>
  </body>
</html>
```

Po linii statusu można zauważyć, że serwer nie wspiera tak starej wersji HTTP. Gdyby tak było, serwer powinien w odpowiedzi podać wersję HTTP/0.9.

Zobaczmy na inny, mniej egzotyczny (choć HTTP/1.0 to również raczej relik przeszłości) przypadek, również korzystając z domeny example.com.

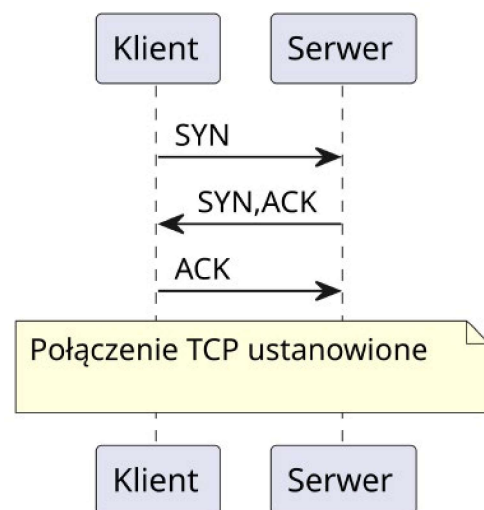
I HTTP a połączenia TCP

Zanim powiemy sobie o utrzymywaniu połączeń w HTTP wersji 1.0 i 1.1, a także o nagłówku Connection, musimy wspomnieć o połączeniach TCP, bo właśnie z tego protokołu korzysta pod spodem HTTP.

TCP jest protokołem, który zapewnia gwarancję dostarczenia danych do odbiorcy. Proces ten polega na tym, że odbiorca wysyła informację zwrotną do nadawcy, potwierdzając odbiór wiadomości. Potwierdzenie to jest wysyłane w pakiecie z ustawioną flagą ACK (acknowledge). Choć ta cecha zapewnia poprawne dostarczenie danych, dodatkowe potwierdzenia powodują nadmierny ruch w sieci i prowadzą do większych opóźnień w komunikacji.

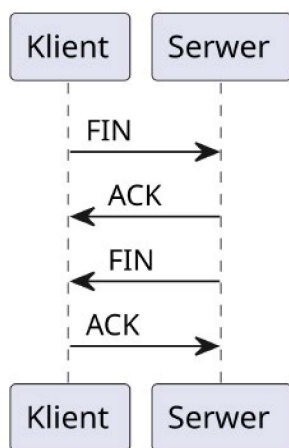
Wiedząc to, możemy iść krok dalej i opowiedzieć sobie o nawiązywaniu i zrywaniu połączeń TCP. Nawiązywanie połączenia odbywa się przez tzw. *three-way-handshake*, a to dlatego, że składa się z trzech pakietów.

Pierwszy pakiet od klienta do serwera (czyli od inicjującego połączenie) to pakiet SYN (synchronize). Jest wykorzystywany właśnie do nawiązania połączenia. Następnie serwer odsyła pakiet SYN, równocześnie informując, że poprawnie odebrał poprzednią ramkę (ACK). Ostatecznie klient potwierdza odebranie pakietu (ACK). W ten sposób zarówno klient, jak i serwer mają nawiązane połączenie – Rysunek 12.



Rysunek 12. Schemat nawiązywania połączenia TCP między klientem a serwerem

Zakończenie połączenia jest bardziej złożoną operacją. W normalnych warunkach powinniśmy zobaczyć dwie pary pakietów FIN/ACK – Rysunek 13.



Rysunek 13. Schemat zamykania połączenia TCP

Istnieje jeszcze inny sposób zakończenia połączenia – poprzez wysłanie pakietu RST. Taki pakiet działa na zasadzie „fire and forget” i nie wymaga potwierdzenia od odbiorcy, co czyni go bardziej wydajnym, ale mniej eleganckim rozwiązaniem. Flagi tej używa się zazwyczaj w sytuacjach, gdy konieczne jest sygnalizowanie błędu.

Widzimy więc, że zarówno otwieranie, jak i zamykanie połączenia jest dość skomplikowane, a co za tym idzie pracochłonne. Szczególnie mało wydajne jest rozwiązanie, w którym nawiązywanie połączenia jest robione per zapytanie HTTP, jeśli klient zamierza tych zapytań wysłać wiele (nawet najprostsze witryny internetowe wysyłają wiele zapytań o zasoby do jednego serwisu), dlatego protokół HTTP ma mechanizm, który pozwala informować strony połączenia o tym, czy połączenie TCP należy utrzymywać, czy też zakończyć.

Wiedząc to, możemy powiedzieć sobie o nagłówku Connection, a także spojrzeć na połączenie oczami Wiresharka.

Warto wspomnieć o jednej z istotnych różnic między HTTP/1.0 a HTTP/1.1, dotyczącej sposobu utrzymywania połączenia. W HTTP/1.0 połączenie było domyślnie zrywane po zakończeniu transakcji, czyli po otrzymaniu odpowiedzi przez klienta. Aby temu zapobiec, należało wysłać nagłówek Connection: keep-alive. Inaczej jest w wersji HTTP/1.1. Tutaj domyślnie połączenie jest utrzymywane, jeśli chcemy zamknięcia połączenia, wysyłamy nagłówek Connection: close.

Warto dodać, że nagłówek ten może być wysłany zarówno przez klienta, jak i serwer.

Zobaczmy to zachowanie w akcji. Weźmy na tapet protokół HTTP/1.0 oraz użyjmy narzędzia ncat.

Listing 12. Użycie ncat w celu wysłania zapytania HTTP

```
$ ncat example.com 80
>GET / HTTP/1.0
>Host: example.com
>
HTTP/1.0 200 OK
Connection: close
<reszta odpowiedzi>
GET / HTTP/1.0
Host: example.com
<error> Ncat: Broken pipe
```

Zerknijmy na kolejne pakiety wysyłane przez aplikację: nawiązanie połączenia SYN → SYN,ACK → ACK (Rysunek 14).

Kolejno zobaczymy 3 pakiety TCP wraz z odpowiadającymi im pakietami ACK:

- » Pakiet TCP zawierający pierwszą linię zapytania (Rysunek 15). Jak widzimy, program ncat wysła dane linia po linii. Można zauważyć również, że znak nowej linii w zapytaniu to szesnastkowo 0x0a, co odpowiada znakowi \n. Poprawną reprezentacją nowej linii w HTTP jest natomiast sekwencja znaków \r\n. Jeśli chcemy naprawić ten problem, powinniśmy wywołać program ncat z flagą -C, która zamienia znaki nowej linii na sekwencje \r\n.
- » Pakiet zawierający nagłówek (Rysunek 16).
- » Ostatnia część, czyli brakujący znak nowej linii, nie jest pokazana jako oddzielny pakiet TCP. Razem z wysłaniem ostatniego znaku Wireshark orientuje się, że to, co wysłaliśmy, jest pakietem HTTP, dlatego następane, co zobaczymy, to właśnie zanalizowany pakiet. Jeśli przyjrzymy się zrzutowi ekranu, to zobaczymy tam adnotację TCP PDU reassembled in 102. PDU znaczy tutaj Protocol Data Unit. W tym przypadku protokół to HTTP. Oznacza to, że we wpisie numer 102 zobaczymy wszystkie zebrane pakiety TCP razem z informacją o wiadomości wyższego poziomu – HTTP.
- » Zanalizowany pakiet (Rysunek 17). W zależności od tego, którą część protokołu wybierzemy, w części binarnej widoku aplikacji zobaczymy albo całe zapytanie HTTP, albo tylko ostatni pakiet TCP zapytania, który został wysłany (Rysunek 18).

W widoku widzimy również wpis 3 reassembled TCP segments zawierający informacje o wszystkich trzech pakietach TCP wysłanych do tej pory. Ostatecznie mamy zebranie wszystkiego w zapytanie HTTP (Rysunek 19).

- » Serwer wysłał nam odpowiedź (w przeciwieństwie do aplikacji ncat robi to bardziej wydajnie i wysła całą odpowiedź w jednym pakiecie TCP) – szczegóły pakietu pozostawiam do analizy we własnym zakresie.
- » Zakończenie połączenia – serwer w odpowiedzi HTTP zapowiedział zakończenie połączenia TCP (nagłówek Connection z wartością close). Jak widzimy, serwer dotrzymał słowa, ponieważ zaraz po wysłaniu odpowiedzi rozpoczął procedurę zamknięcia połączenia (Rysunek 20).

Warto przypomnieć jedną rzecz – HTTP jest protokołem warstwy aplikacji, podczas gdy TCP jest protokołem warstwy transportu danych. TCP jako taki jest głównie obsługiwany przez system operacyjny, podczas gdy HTTP przez aplikację. Zakończenie połączenia na poziomie warstwy TCP nie jest widoczne przez aplikację. Dlatego też w protokole HTTP serwer uprzedza o chęci zamknięcia połączenia. Nie widzimy jednak drugiej pary FIN, ACK, ponieważ nasza aplikacja zignorowała chęć zamknięcia połączenia.

- » W naszym przypadku zignorowaliśmy nagłówek Connection: close i próbowaliśmy wysłać kolejny pakiet danych, który zakończył się dla nas błędem – broken pipe. Zobaczmy, co wydarzyło się w warstwie TCP – Rysunek 21.

Widzimy więc, że ncat wysłał dane do serwera, jednak ten nie rozpoznawszy połączenia (które zostało zamknięte), odsyła pakiet RST, sygnalizujący błąd. Dopiero teraz aplikacja orientuje się, że wystąpił problem przy wysyłaniu danych i raportuje go użytkownikowi.

No.	Time	Source	Destination	Protocol	Length	Info
14	5.003339	10.11.0.25	93.184.215.14	TCP	74	58032 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2696405146 TSecr=0 WS=128
15	5.103145	93.184.215.14	10.11.0.25	TCP	74	80 → 58032 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=3665794575 TSecr=2
16	5.103191	10.11.0.25	93.184.215.14	TCP	66	58032 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2696405246 TSecr=3665794575

Rysunek 14. Nawiązanie połączenia widziane w aplikacji Wireshark

89	23.67202	10.11.0.25	93.184.215.14	TCP	81	58032 → 80 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=15 TSval=2696423814 TSecr=3665794575 [TCP
90	23.77401	93.184.215.14	10.11.0.25	TCP	66	80 → 58032 [ACK] Seq=1 Ack=16 Win=65535 Len=0 TSval=3665813246 TSecr=2696423814
0000	9e 05 d6 53 89 8f 74 e5	f9 86 f9 0c 08 00 45 00	...S...t...E			
0010	00 43 09 84 40 00 40 06	f2 46 0a 0b 00 19 5d b8	...C...@...F...			
0020	d7 0e e2 b0 00 50 12 59	29 21 70 3c 6f 8a 80 18	...P...Y...Jpco...			
0030	01 f6 2e 2a 00 00 01 01	08 0a a0 b8 29 86 da 7f	...S;...V...			
0040	96 0f 47 45 54 20 2f 20	48 54 54 50 2f 31 2e 30	...Host: example.com			
0050	0a		GET / HTTP/1.0			

Rysunek 15. Pakiet TCP z pierwszą linią zapytania

No.	Time	Source	Destination	Protocol	Length	Info
127	35.05840	10.11.0.25	93.184.215.14	TCP	84	58032 → 80 [PSH, ACK] Seq=16 Ack=1 Win=64256 Len=18 TSval=2696435201 TSecr=3665813246 [TC
130	35.16054	93.184.215.14	10.11.0.25	TCP	66	80 → 58032 [ACK] Seq=1 Ack=34 Win=65536 Len=0 TSval=3665824633 TSecr=2696435201
132	36.11856	10.11.0.25	93.184.215.14	HTTP	67	GET / HTTP/1.0
133	36.21799	93.184.215.14	10.11.0.25	TCP	66	80 → 58032 [ACK] Seq=1 Ack=35 Win=65536 Len=0 TSval=3665825691 TSecr=2696436261
134	36.22128	93.184.215.14	10.11.0.25	HTTP	1698	HTTP/1.0 200 OK (text/html)
0000	9e 05 d6 53 89 8f 74 e5	f9 86 f9 0c 08 00 45 00	...S...t...E			
0010	00 46 09 85 40 00 40 06	f2 42 0a 0b 00 19 5d b8	...F...@...B...			
0020	47 0e e2 b0 00 50 12 59	29 20 70 3c 6f 8a 00 10	...P...Y...Opco...			
0030	01 f6 24 3b 00 00 01 01	08 0a a0 b8 56 01 da 7f	...S;...V...			
0040	de fe 48 6f 73 74 3a 20	65 78 61 6d 70 6c 65 2e	...Host: example			
0050	63 6f 6d 0a		com.			

Rysunek 16. Pakiet TCP z nagłówkiem

132	36.1185	10.11.0.25	93.184.215.14	HTTP	67	GET / HTTP/1.0
133	36.2179	93.184.215.14	10.11.0.25	TCP	66	80 → 58032 [ACK] Seq=1 Ack=35 Win=65536 Len=0 TSval=3665825691 TSecr=2696436261

Rysunek 17. Zapytanie HTTP

Frame 132: 67 bytes on wire (536 bits), 67 bytes captured (536 bits) on interface wlp2s0, id 0		0000	9e 05 d6 53 89 8f 74 e5	f9 86 f9 0c 08 00 45 00	...S...t...E
Ethernet II, Src: Intel_86:f9:0c (74:e5:f9:86:f9:0c), Dst: 9e:05:d6:53:89:8f		0010	00 35 09 86 40 00 40 06	f2 52 0a 0b 00 19 5d b8	...5...@...R...
Internet Protocol Version 4, Src: 10.11.0.25, Dst: 93.184.215.14		0020	d7 0e e2 b0 00 50 12 59	29 42 70 3c 6f 8a 80 18	...P...Y...Jpco...
Transmission Control Protocol, Src Port: 58032, Dst Port: 80, Seq: 34, Ack: 1		0030	01 f6 4c 99 00 00 01 01	08 0a a0 b8 5a 25 da 80	...L...Z...
[3 Reassembled TCP Segments (34 bytes): #89(15), #127(18), #132(1)]		0040	0b 79 0a		...Y...
Hypertext Transfer Protocol					

Rysunek 18. Ostatnia część zapytania jako pakiet TCP

Frame 102: 67 bytes on wire (536 bits), 67 bytes captured (536 bits) on interface wlp2s0, id 0		0000	47 45 54 20 2f 20 48 54	54 50 2f 31 2e 30 0a 48	GET / HTTP/1.0-H
Ethernet II, Src: Intel_86:f9:0c (74:e5:f9:86:f9:0c), Dst: 9e:05:d6:53:89:8f (9e:05:d6:53:89:8f)		0010	6f 73 74 3a 20 65 78 61	6d 70 6c 65 2e 63 6f 6d	ost: example.com
Internet Protocol Version 4, Src: 10.11.0.25, Dst: 93.184.215.14		0020	0a 0a		...
Transmission Control Protocol, Src Port: 60792, Dst Port: 80, Seq: 34, Ack: 1, Len: 1					
[3 Reassembled TCP Segments (34 bytes): #95(15), #99(18), #102(1)]					
Hypertext Transfer Protocol					

Rysunek 19. Szczegóły żądania HTTP w aplikacji Wireshark

135	36.221	93.184.215.14	10.11.0.25	TCP	66	80 → 58032 [FIN, ACK] Seq=1633 Ack=35 Win=65536 Len=0 TSval=3665825693
136	36.221	10.11.0.25	93.184.215.14	TCP	66	58032 → 80 [ACK] Seq=35 Ack=1633 Win=62720 Len=0 TSval=2696436364 TSecr=3665825693
138	36.263	10.11.0.25	93.184.215.14	TCP	66	58032 → 80 [ACK] Seq=35 Ack=1634 Win=62720 Len=0 TSval=2696436406 TSecr=3665825693

Rysunek 20. Zamknięcie połączenia TCP widziane w aplikacji Wireshark

212	57.669	10.11.0.25	93.184.215.14	TCP	81	58032 → 80 [PSH, ACK] Seq=35 Ack=1634 Win=62720 Len=15 TSval=2696457812
213	57.769	93.184.215.14	10.11.0.25	TCP	56	80 → 58032 [RST] Seq=1634 Win=0 Len=0

Rysunek 21. Zignorowanie zamknięcia połączenia i dalsze wysłanie danych oraz odpowiedź RST

Podobny przykład, ale z podtrzymaniem połączenia między klientem a serwisem, widzimy w Listingu 13.

Listing 13. Żądanie HTTP 1/1 wysłane z aplikacji nc

```
$ncat -C example.com 80
>GET / HTTP/1.1
>Host: example.com

HTTP/1.1 200 OK
<Brak nagłówka Connection - domyślnie keep-alive>
<ciało odpowiedzi>
>GET / HTTP/1.1
>Host: example.com

HTTP/1.1 200 OK
...
```

Korzystając z wersji HTTP/1.1, domyślnie informujemy serwer, że połączenie powinno pozostać otwarte. W rezultacie serwer nie wysłał próśb o jego zamknięcie. Przy wysłaniu kolejnego zapytania nie pojawiają się błędy, a zamiast tego dostajemy poprawną odpowiedź.

Analizę tego przypadku zostawiam czytelnikowi.

I PROXY – MITMPROXY

Zanim przejdziemy do omawiania pozostałych funkcji protokołu, zatrzymajmy się na chwilę przy narzędziu, które ułatwi nam pracę z aplikacjami komunikującymi się za pomocą HTTP. Mowa o mitmproxy, które osobiście bardzo cenię za ułatwianie mojej codziennej pracy. Oczywiście nie jest to narzędzie jedyne w swoim rodzaju, bo istnieją podobne, jak choćby BurpSuite czy ZAP, w szczególności używane przez osoby zajmujące się dziedziną cyberbezpieczeństwa.

Jak sama nazwa wskazuje, jest to aplikacja typu proxy. Możemy za jego pomocą przechwytywać, analizować, przekierowywać, a nawet modyfikować zapytania w czasie rzeczywistym. Ponadto mitmproxy udostępnia API w języku Python, które pozwala na samodzielne rozszerzanie funkcjonalności aplikacji (gdyby się okazało, że jego wbudowane cechy nie są wystarczające).

Mitmproxy oferuje interfejs linii poleceń (polecenie mitmproxy), interfejs aplikacji webowej (polecenie mitmweb), a także nieinteraktywną wersję swoich funkcjonalności (polecenie mitmdump).

Przyjrzyjmy się kilku scenariuszom użycia mitmproxy i zobaczmy, jak narzędzie to sprawdza się w praktyce.

I Podglądanie ruchu sieciowego

Cały ruch sieciowy związany z daną maszyną można podglądać w aplikacji Wireshark. Jest to niezwykle wygodne – wystarczy uruchomić aplikację i już możemy oglądać kolejne wysyłane i odbierane pakiety danych.

Głównym problemem tego podejścia jest to, że obecnie większość ruchu sieciowego jest szyfrowana. Niektóre aplikacje pozwalają na eksport danych umożliwiających odszyfrowanie ruchu, a często konfiguracja taka jest po prostu niewygodna (trzeba ustawiać zmienne środowiskowe i zrestartować aplikację).

Praktycznie zawsze nasze środowisko pozwala na użycie proxy. Niektóre aplikacje pozwalają zdefiniować proxy wprost (np. przeglądarki poprzez swoje ustawienia, a aplikacja curl przez parametr ko-

mendy). Jeszcze inne możemy zmusić do używania proxy np. przez aplikację proxychains.

Napiszmy prosty program w języku JavaScript:

Listing 14. Prosta aplikacja w języku JavaScript komunikująca się z użyciem HTTP

```
setInterval(async () => {
  const response = await fetch('https://example.com')
  console.log(response.status)
}, 1000)
```

Aplikacja ta co sekundę wysyła żądanie do domeny example.com i wyświetla status odpowiedzi na konsolę. W aplikacji Wireshark nie zobaczymy szczegółów komunikacji, ponieważ będzie ona zaszyfrowana (URI do funkcji fetch ma zdefiniowany protokół HTTPS). Użyjmy teraz aplikacji proxychains, aby przekierować żądanie do proxy, tym samym umożliwiając ingerencję w komunikację.

Po pierwsze, uruchomimy aplikację mitmproxy: mitmproxy --mode socks5@5000 – w ten sposób mitmproxy będzie nasłuchiwało ruch na porcie 5000 i komunikowało się z użyciem protokołu socks5.

Następnie utworzymy prosty plik konfiguracyjny do aplikacji proxychains:

Listing 15. Prosta konfiguracja aplikacji proxychains

```
strict_chain
[ProxyList]
socks5 127.0.0.1 5000
```

Brakuje jeszcze jednej rzeczy. Ponieważ mitmproxy podsłuchuje komunikację, certyfikaty szyfrowania nie będą mogły być zweryfikowane (certyfikat widziany w aplikacji JavaScript będzie nieznanym). Domyślnie node.js nie pozwala na komunikację z niezaufanymi serwisami. Aby temu zapobiec, musimy zmusić node.js do akceptacji certyfikatu (podobnie należy postąpić z dowolną aplikacją, której szyfrowany ruch będziemy podsłuchiwać). Aby to zrobić, należy ustawić zmienną środowiskową NODE_EXTRA_CA_CERTS.

Wystartujmy nasz program:

Listing 16. Komenda uruchamiająca aplikację w node.js

```
NODE_EXTRA_CA_CERTS=~/.mitmproxy/mitmproxy-ca-cert.cer\
proxychains -f proxy.conf node ./app.js
```

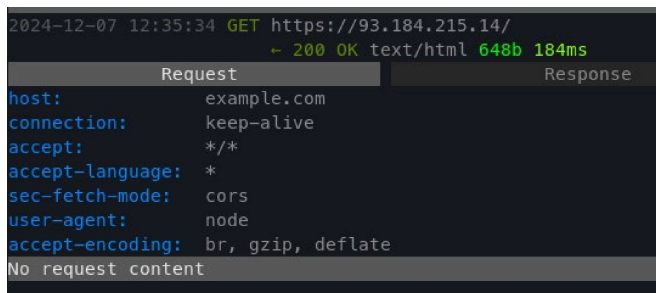
W naszym proxy powinniśmy teraz obserwować kolejne zapytania wysyłane przez naszą aplikację:

```
>>12:35:34 HTTPS GET 93.184.215.14 / 200 text/html 648b 182ms
12:35:34 HTTPS GET 93.184.215.14 / 200 text/html 648b 184ms
12:35:35 HTTPS GET 93.184.215.14 / 200 text/html 648b 204ms
12:35:36 HTTPS GET 93.184.215.14 / 200 text/html 648b 231ms
```

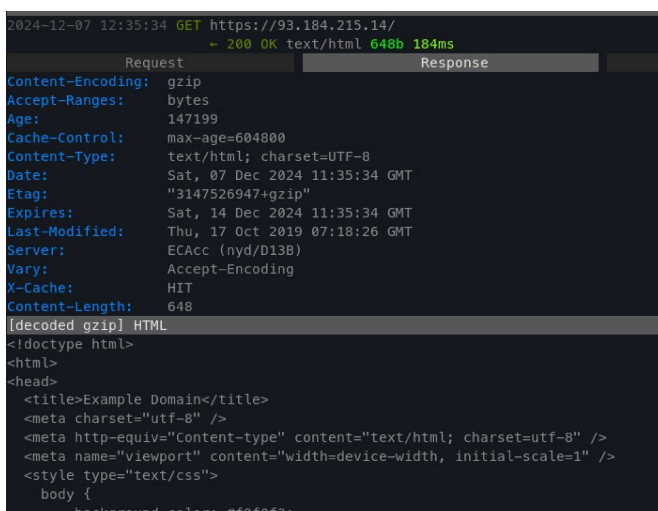
Rysunek 22. Kolejne żądania pojawiające się w aplikacji mitmproxy

W aplikacji mamy wgląd we wszystkie szczegóły zapytania po wybraniu odpowiedniej wiadomości. Możemy podejrzeć zarówno żądanie – jego pierwszą linię, nagłówki oraz ciało, tak samo jak i linię statusu, nagłówki oraz ciało odpowiedzi. Ponadto w zakładce Detail zoba-

czymy informacje związane z umiejscowieniem zapytania i żądania w czasie. Przykładowy podgląd szczegółów zapytania i odpowiedzi znajdziemy odpowiednio na Rysunkach 23 oraz 24.



Rysunek 23. Podgląd szczegółów zapytania w aplikacji mitmproxy



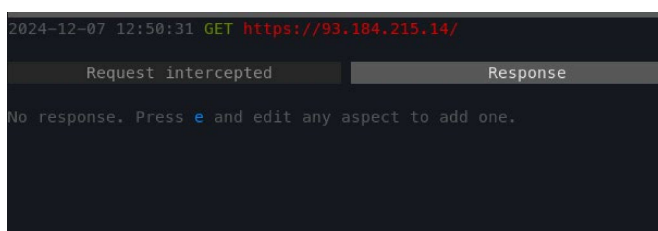
Rysunek 24. Podgląd szczegółów odpowiedzi w aplikacji mitmproxy

I Modyfikacja ruchu sieciowego na żywo

Mitmproxy pozwala nie tylko na podglądanie ruchu sieciowego, ale również na jego modyfikację. Korzystając z poprzednich ustawień, możemy modyfikować ruch ręcznie, lub automatycznie korzystając z odpowiednich ustawień.

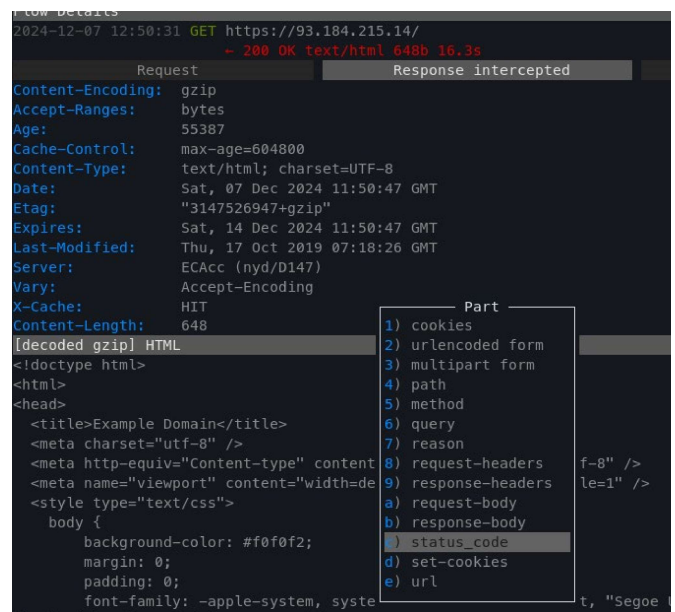
W mitmproxy pod opcją i możemy modyfikować wyrażenie filtrujące, które definiuje, jakie wiadomości będą modyfikowane. Wpiszmy sobie więc wyrażenie `~all`, które przechwyci wszystkie wiadomości, a następnie zmodyfikujmy jedną z odpowiedzi wysłanych z serwera do klienta.

W pierwszym kroku wybieramy jedno z żądań, które zostały podświetlone na kolor czerwony – Rysunek 25.

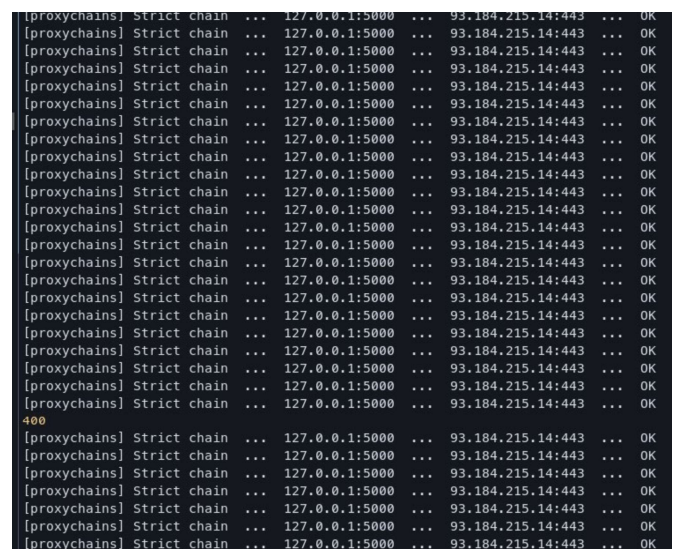


Rysunek 25. Wybór przechwyconego do modyfikacji żądania HTTP

Kolejno wciskamy przycisk **a**, który spowoduje wysłanie żądania do serwera. Powinniśmy zobaczyć odpowiedź z serwera, którą możemy modyfikować (przed przesłaniem jej do klienta). Następnie naciskamy przycisk **e**, aby przejść do edycji, i wybieramy opcję `status_code`. Zmienimy kod statusu na 400 i obserwujemy wyjście aplikacji (na naszej konsoli powinno pojawić się `400`).



Rysunek 26. Wybór części odpowiedzi do modyfikacji



Rysunek 27. Wyjście aplikacji po wysłaniu zmodyfikowanej odpowiedzi

Magiczne wyjścia pojawiające się w naszej aplikacji zaczynające się od [proxychains] możemy zignorować, ponieważ świadczą one o nawiązywaniu nowych połączeń z serwerem.

Po pewnym czasie aplikacja zgłosi błąd żądania „timeout”, ponieważ upłynęło zbyt wiele czasu do momentu uzyskania odpowiedzi (aby do tego nie doszło, musielibyśmy wznowić wszystkie przechwycone do modyfikacji zapytania).

Możemy też automatycznie modyfikować części komunikacji, np. dodając lub modyfikując nagłówki. Przykład takiego wywołania aplikacji mitmproxy znajduje się w Listingu 17.

Listing 17. Uruchomienie proxy dodającego nagłówki `used-proxy` do każdego żądania

```
mitmproxy --mode socks5@5000 -H '|~q|used-proxy|mitmproxy'
```

Takie wywołanie spowoduje, że do każdego żądania (~q) zostanie dodany nagłówek o kluczu `used_proxy` i wartości `mitmproxy`, co widzimy na ekranie:

Flow Details	
2024-12-07 13:17:56 GET https://93.184.215.14/	
← 200 OK text/html 648b 151ms	
Request	Response
host: example.com connection: keep-alive accept: */* accept-language: * sec-fetch-mode: cors user-agent: node accept-encoding: br, gzip, deflate used-proxy: mitmproxy No request content	

Rysunek 28. Przykład żądania HTTP z dodanym nagłówkiem `used-proxy`

Podobnie do modyfikacji nagłówków możemy zmienić serwis, do którego trafiają żądania. Może to być przydatne, jeśli np. potrzebujemy przetestować aplikację webową z hostowanym lokalnie serwerem. Możemy tego dokonać za pomocą opcji `--map_remote`:

Listing 18. Przykład użycia `mitmproxy` przekierowującego żądania z domeny `example.com` do domeny `example1.com`

```
mitmproxy --mode socks5@5000 --map-remote '|example.com|example1.com'
```

W widoku otrzymamy:

Flows	
13:23:12	HTTPS GET example1.com / 200 text/html 2.2k 452ms
13:23:13	HTTPS GET example1.com / 200 text/html 2.2k 428ms
13:23:13	HTTPS GET example1.com / 200 text/html 2.2k 158ms
13:23:14	HTTPS GET example1.com / 200 text/html 2.2k 206ms
13:23:15	HTTPS GET example1.com / 200 text/html 2.2k 154ms
13:23:16	HTTPS GET example1.com / 200 text/html 2.2k 250ms
13:23:17	HTTPS GET example1.com / 200 text/html 2.2k 169ms
13:23:18	HTTPS GET example1.com / 200 text/html 2.2k 297ms
>>13:23:19	HTTPS GET example1.com / 200 text/html 2.2k 214ms
13:23:20	HTTPS GET example1.com / 200 text/html 2.2k 221ms
13:23:21	HTTPS GET example1.com / 200 text/html 2.2k 263ms
13:23:22	HTTPS GET example1.com / 200 text/html 2.2k 189ms

Rysunek 29. Wynik wywołania Listingu 18

Flow Details	
2024-12-07 13:23:19 GET https://example1.com/	
← 200 OK text/html 2.2k 214ms	
Request	Response
Date: Sat, 07 Dec 2024 12:23:20 GMT Server: Apache Last-Modified: Tue, 15 Sep 2020 14:51:09 GMT Accept-Ranges: bytes Vary: Accept-Encoding Content-Encoding: gzip Content-Length: 2255 Keep-Alive: timeout=5, max=68 Connection: Keep-Alive Content-Type: text/html [decoded gzip] HTML	

Rysunek 30. Wynik wywołania Listingu 18

Mam nadzieję, że te kilka przykładów przekonało czytelnika o uniwersalności tego narzędzia. Oczywiście to narzędzie ma o wiele więcej funkcjonalności niż tutaj opisano, dlatego serdecznie zachęcam do odwiedzenia jego strony internetowej [26] i dokładniejszego się z nim zapoznania.

I CACHE

Niektórzy uczestnicy komunikacji HTTP mogą posiadać swój *cache*, który dzielimy na dwa rodzaje: prywatny oraz publiczny.

Cache prywatny znajduje się blisko użytkownika. Może to być np. *cache* przeglądarki internetowej lub aplikacja mobilna w telefonie.

Cache publiczny (współdzielony, ang. *shared*) serwuje dane dla wielu użytkowników. Zazwyczaj jest on implementowany po stronie serwisu internetowego.

Rozróżnienie w specyfikacji HTTP na te dwa rodzaje buforów ma sprzyjać zachowaniu prywatności użytkownika. Dane, które należą do konkretnego użytkownika, przechowywane w publicznych cache'ach mogłyby naruszyć prywatność użytkowników.

Zachowanie buforów sterujemy nagłówkiem `Cache-Control`. Wartość tego nagłówka składa się zazwyczaj z kilku składowych dyrektyw oddzielonych od siebie przecinkiem.

Jedną z tych dyrektyw jest właśnie `private` lub `public`. `Private` oznacza, że odpowiedź powinna być przechowywana wyłącznie w prywatnym *cache*, a `public`, że może być przechowywana również w publicznym *cache*.

I Świeżość odpowiedzi

Kiedy *cache* uznaje przechowywaną odpowiedź za świeżą, serwuje ją użytkownikowi, nie walidując jej w żaden sposób z origin serwerem.

Serwer wysyłając nagłówek `Cache-Control`, informuje o tym, jak długo odpowiedź może być uznawana za świeżą, poprzez dyrektywę `max-age` (dla *cache*'a prywatnego) oraz `s-maxage` dla *cache*'a współdzielonego. Argument dyrektyw określa, jak długo odpowiedź jest świeża (w sekundach), np. `Cache-Control: max-age=60` oznacza, że odpowiedź jest świeża przez minutę.

Kiedy upłynie czas świeżości przechowywanego elementu, oznaczony jest on jako „stałe” (czerstwy), natomiast nie jest on jeszcze usuwany. Zamiast tego, przy kolejnym żądaniu, *cache* wyśle specjalne zmodyfikowane żądanie do serwisu, który odeśle ciało odpowiedzi tylko wtedy, kiedy ciało przechowane w *cache*'u nie będzie aktualne. W ten sposób oszczędzamy przesył danych.

Z ciekawostek, istnieją dyrektywy jak np. `stale-if-error` lub `max-stale`, które pozwalają na użycie takiego nieświeżego elementu.

Oczywiście, możemy w ogóle nie chcieć *cache*ować odpowiedzi. Często zobaczymy w odpowiedziach dyrektywę `no-cache`, która zdawać by się mogło służy właśnie do tego. Niestety, jest to myląca nazwa. Takie odpowiedzi nadal będą przechowywane w *cache*'u, ale natychmiast będą oznaczane jako nieświeże. Dyrektywa `no-cache` to odpowiednik `max-age=0`;

Aby zapobiec jakimkolwiek przechowywaniu odpowiedzi w *cache*'u (np. ze względu na wysoki stopień prywatności danych), należy użyć dyrektywy `no-store`.

I Odświeżanie odpowiedzi i żądania warunkowe

Kiedy już dojdzie do tego, że odpowiedź będzie stara, a *cache* będzie chciał ją wykorzystać, zostanie wysłane do serwisu wspomniane już żądanie warunkowe [28].

Trzeba wiedzieć, że *cache* nie tylko korzysta z informacji o tym, jak długo odpowiedź ma być świeża, ale dodatkowo korzysta z wartości dwóch innych nagłówków: *Etag* oraz *Last-Modified*.

Żądania warunkowe to takie, które serwis przetwarza tylko pod pewnymi warunkami. Zachowanie to sterowane jest nagłówkami, zwłaszcza:

- » *If-None-Match*
- » *If-Match*
- » *If-Unmodified-Since*
- » *If-Modified-Since*

If-None-Match oraz *If-Modified-Since* są używane do sprawdzenia, czy nowsza wersja zasobu znajduje w serwisie, i to właśnie one są wykorzystywane przez *cache*. *If-None-Match* korzysta z wartości nagłówka *Etag*, a *If-Modified-Since* z *Last-Modified*.

Spójrzmy na przykład takiego zapytania. Pierwsze zapytanie o zasób będzie wyglądać następująco:

Listing 19. Użycie aplikacji curlie do wysłania żądania bezwarunkowego

```
$curlie -v -I http://example.com

HTTP/1.1 200 OK
Content-Encoding: gzip
Accept-Ranges: bytes
Age: 221877
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Sat, 07 Dec 2024 15:05:44 GMT
Etag: "3147526947"
Expires: Sat, 14 Dec 2024 15:05:44 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECAcc (nyd/D188)
X-Cache: HIT
Content-Length: 648
```

Widzimy w odpowiedzi zarówno nagłówek *Etag*, jak i *Last-Modified*. *Last-Modified* jest łatwy do interpretacji, jest to bowiem data ostatniej modyfikacji, zapisana w raczej przystępny sposób. Jeśli chodzi o wartość *Etag*, to jest to wyliczana, unikalna wartość identyfikująca zasób w danej wersji.

Żeby sprawdzić, czy nasza odpowiedź jest aktualna, wystarczy zapytać, czy zasób został zmodyfikowany od naszej podanej daty w nagłówku. Zweryfikujmy to:

Listing 20. Żądanie warunkowe z wykorzystaniem nagłówka *If-Modified-Since*

```
$curlie http://example.com \
If-Modified-Since: 'Thu, 17 Oct 2019 07:18:26 GMT'

HTTP/1.1 304 Not Modified
Accept-Ranges: bytes
Age: 159579
Cache-Control: max-age=604800
Date: Sat, 07 Dec 2024 15:02:58 GMT
Etag: "3147526947+gzip"
Expires: Sat, 14 Dec 2024 15:02:58 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECAcc (nyd/D136)
X-Cache: HIT
```

Jak widzimy, otrzymaliśmy odpowiedź 304 – not modified. Oznacza to, że zasób nie został zmodyfikowany od tego czasu. W takim przypadku nie otrzymujemy od serwisu ciała z zasobem.

Podobnie sprawa się ma z nagłówkiem *Etag*. Żądanie będzie wyglądać następująco:

Listing 21. Żądanie warunkowe z wykorzystaniem nagłówka *If-None-Match*

```
$curlie -v -I http://example.com 'If-None-Match: "3147526947"'

HTTP/1.1 304 Not Modified
Accept-Ranges: bytes
Age: 221879
Cache-Control: max-age=604800
Date: Sat, 07 Dec 2024 15:05:46 GMT
Etag: "3147526947"
Expires: Sat, 14 Dec 2024 15:05:46 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECAcc (nyd/D188)
X-Cache: HIT
```

Podobnie i w tym przypadku otrzymujemy informację o tym, że zasób nie został zmodyfikowany.

I Unikanie kolizji

Skoro jesteśmy w temacie żądań warunkowych, powiedzmy sobie jeszcze krótko o dwóch pozostałych nagłówkach, czyli *If-Unmodified-Since* oraz *If-Match*. Są to warunki zupełnie odwrotne w zachowaniu od tych, które właśnie omawialiśmy.

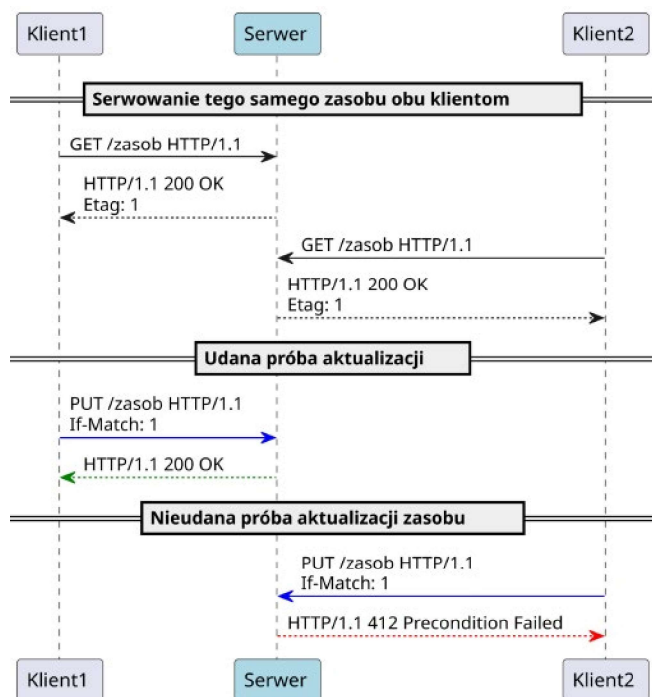
Można zadać sobie pytanie, po co nam tego typu żądania. Okazuje się, że są bardzo przydatne, kiedy aktualizujemy jakiś zasób (np. metodą *PUT*) i chcemy mieć pewność, że w międzyczasie nikt inny tego zasobu nie zaktualizował. Innymi słowy, chcemy, aby aktualizacje nie kolidowały ze sobą.

..... / * REKLAMA * /

ARCHIWALNE WYDANIA PROGRAMISTY
W E-BOOK POINT:
[HTTPS://EBOOKPOINT.PL/AUTORZY/MAGAZYN-PROGRAMISTA](https://ebookpoint.pl/autorzy/magazyn-programista)

Inaczej również niż w poprzednim przypadku jest sygnalizowana niezgodność z przekazanym warunkiem. Wtedy otrzymywaliśmy odpowiedź 304, w tym przypadku jednak standardową odpowiedzią jest 412 Precondition Failed.

Zobaczmy przykład z użyciem nagłówka If-Match – Rysunek 31.



Rysunek 31. Scenariusz z użyciem nagłówka If-Match

Jak widać, w tym przypadku mamy 2 użytkowników, chcących zaktualizować jeden zasób. Chcemy uniknąć sytuacji, gdzie któraś z aktualizacji jest całkiem pominięta i nadpisana, dlatego właśnie używamy wspomnianego mechanizmu unikania kolizji. Klientowi 2 nie udaje się zaktualizować zasobu, ponieważ wartość Etag zmieniła się po aktualizacji zasobu przez klienta 1.

RÓŻNICE MIĘDZY WERSJAMI PROTOKOŁU HTTP/1.1 A JEGO NASTĘPCAMI

Wiemy już, że istnieją różne wersje protokołu HTTP, w tym jego nowsze odsłony, jak HTTP 2 czy nawet HTTP 3. Warto wspomnieć o kilku bolączkach protokołu i o ich rozwiązaniu w następnych wersjach.

Protokół tekstowy a binarny

Przede wszystkim protokół HTTP to protokół tekstowy, co od razu stawia jego wydajność pod znakiem zapytania. Różnego rodzaju wartości liczbowe, jak chociażby te w nagłówku Content-Length czy informacje o liczbie bajtów części odpowiedzi przy Transfer-Encoding: chunked, nie są wysyłane w optymalny sposób.

Oczywiście ma to swoje plusy – możemy bez problemu podglądać wiadomości HTTP czy tworzyć je „z palca” w takich aplikacjach jak chociażby nc.

Dodatkowo wprowadza to pewną niespójność między parserami. Niektóre parsery np. obsługują znak nowej linii jako \n, a inne nie, co mieliśmy okazję obserwować w tym artykule przy okazji omawiania aplikacji nc.

HTTP2 jest już protokołem binarnym, co rozwiązuje powyższe problemy.

Kompresja danych

Kolejny problem wydajnościowy wiąże się z kompresją danych. W HTTP/1.1 możemy skompresować ciało żądania oraz odpowiedzi i oznaczamy to w odpowiednim nagłówku Content-Encoding: gzip (istnieją inne opcje niż gzip). O ile jest to wydajne rozwiązanie, to kompresji takiej nie przewiduje się co do samych nagłówków, co również niepotrzebnie odbija się na rozmiarze wysyłanej przez sieć wiadomości.

HTTP2 potrafi skompresować nagłówki poprzez algorytm HPACK. Lepsza kompresja to mniej danych do przesyłu, a to koreluje z szybkością przesyłu wiadomości po sieci.

Multiplexing

Protokół HTTP/1.1 przewiduje sekwencyjne przetwarzanie żądań klienta. Oznacza to, że jeśli klient będzie chciał np. jednocześnie pobrać 2 zasoby z serwisu, to będzie on musiał otworzyć 2 połączenia z serwerem.

Jest to oczywiście niewydajne. Samo nawiązywanie połączenia wiąże się ze sporym narzutem. Dlatego w HTTP 2 ten problem został naprawiony i aplikacje obsługujące protokół HTTP/2 mogą wysyłać odpowiedzi na wiele żądań z jednego połączenia równocześnie.

Serwer push

Ciekawe rozwiązanie w protokole HTTP 2, które pozwala na wysłanie zasobu klientowi, nawet jeśli ten nie wysłał nigdy o niego żądania. Mechanizm ten opiera się na obserwacji, że niektóre zasoby bardzo często pobierane są jednocześnie (np. plik html, css oraz js serwisu internetowego). Teraz, kiedy serwis otrzyma żądanie o zasób, może mu w odpowiedzi odesłać wiele zasobów.

HTTP 3

To kolejna odsłona protokołu HTTP. Zmienia ona używany „pod maską” protokół transportu. HTTP/2 oraz HTTP/1.1 używały protokołu TCP, aby wiadomości bezpiecznie docierały do adresata. Niestety TCP jest dość „gadatliwym” protokołem. Ponadto HTTPS korzysta również z protokołu TLS do szyfrowania danych, co wymusza wysłanie dodatkowych pakietów w celu uzyskania połączenia TLS.

Odpowiedzią na to jest właśnie HTTP 3, które miałoby się opierać na protokole QUIC, a ten z kolei na protokole UDP. Ideą tego jest używanie na poziomie transportu wydajnego protokołu i przeniesienie pozostałej funkcjonalności do protokołu QUIC, który potrafi w wydajniejszy sposób chociażby nawiązać połączenie TLS.

I Websockets

Chociaż protokół Websockets nie jest następną wersją protokołu HTTP, ani nie ma go zastępować, to warto o nim wspomnieć, chociażby ze względu na jego asynchroniczną naturę.

W tym modelu komunikacji serwer nie musi czekać na żądanie od klienta, aby wysłać mu wiadomość. Po nawiązaniu połączenia serwer może wysłać klientowi wiadomość w dowolnym momencie.

Co więcej, do nawiązania połączenia w tym protokole używa się właśnie protokołu HTTP. Dokładniej, wysyłane jest żądanie o zmianę protokołu.

I PODSUMOWANIE

Oczywiście protokół HTTP jest bardziej rozbudowany niż to, co tutaj zostało zawarte, jednak mam nadzieję, że zainteresowałem czytelnika

tematem i wprowadziłem w jego podstawy na tyle wystarczająco, by resztę wiedzy czytelnik zgłębił we własnym zakresie.

A skoro już o tym mowa, szczególnie polecam zaznaczyć się z tematami:

- » Cookies – czyli o tym, w jaki sposób serwisy zmuszają przeglądarki internetowe do przechowywania i odsyłania im danych.
- » Autoryzacji – wiele serwisów akceptuje żądania HTTP tylko po wcześniejszym ich autoryzowaniu. Klient może przysyłać serwerowi dane autoryzacyjne na wiele sposobów.
- » CORS – mechanizmy, które pozwalają w bezpieczny sposób definiować, które strony internetowe mogą wysyłać żądania i otrzymywać odpowiedzi od naszego serwisu.

W kolejnym, ostatnim już artykule przyjrzymy się nieco bliżej protokołowi TCP oraz UDP oraz poznamy nieco bliżej niektóre z funkcjonalności routerów. Do usłyszenia.

Bibliografia

1. Standard HTTP – <https://www.rfc-editor.org/rfc/rfc7230.html>
3. Wikipedia – <https://pl.wikipedia.org/wiki/HTTP>
4. Pierwszy HTTP – <https://www.w3.org/Protocols/HTTP/AsImplemented.html>
5. Wojny przeglądarek – https://en.wikipedia.org/wiki/Browser_wars
6. User-Agent – https://en.wikipedia.org/wiki/User-Agent_header
7. Architektura HTTP – <https://www.rfc-editor.org/rfc/rfc7230.html#section-2>
8. URL (Uniform Resource Locators) – <https://datatracker.ietf.org/doc/html/rfc1738>
9. URI (Uniform Resource Identifier) – <https://datatracker.ietf.org/doc/html/rfc3986>
10. Percent encoding – <https://datatracker.ietf.org/doc/html/rfc3986#section-2>
11. Accept header – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Accept>
12. Content negotiation – https://developer.mozilla.org/en-US/docs/Web/HTTP/Content_negotiation
13. Transfer encoding, chunked encoding – <https://greenbytes.de/tech/webdav/rfc2616.html#rfc.section.4.4>, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Transfer-Encoding>
14. HTTP methods – <https://www.rfc-editor.org/rfc/rfc7231#section-4>
15. GET – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET>
16. HEAD – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/HEAD>
17. PUT – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/PUT>
18. POST – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST>
19. Idempotentność – <https://pl.wikipedia.org/wiki/Idempotentno%C5%9B%C4%87>
20. One Drive rest API upload – https://learn.microsoft.com/en-us/onedrive/developer/rest-api/api/driveitem_put_content?view=odsp-graph-online
21. One Drive rest API delete – https://learn.microsoft.com/en-us/onedrive/developer/rest-api/api/driveitem_delete?view=odsp-graph-online
22. OPTIONS – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/OPTIONS>
23. Allow header – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Allow>
24. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
25. Keep-alive – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Keep-Alive>
26. <https://mitmproxy.org/>
27. Cache-Control – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Cache-Control>
28. Żądania warunkowe – https://developer.mozilla.org/en-US/docs/Web/HTTP/Conditional_requests
29. CORS, preflight – <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS> https://developer.mozilla.org/en-US/docs/Glossary/Preflight_request
30. Multipart/ form-data – <https://www.rfc-editor.org/rfc/rfc2388>



DAWID PILARSKI

dawid.pilarski@panicsoftware.com

Z wykształcenia automatyk i robotyk, a z zawodu i pasji programista. Obecnie Software Engineer, Security Champion i Tech Lead w TomTom. Wolny czas przeznacza na zgłębianie wiedzy o bezpieczeństwie i sieciach. Można się z nim skontaktować poprzez adres e-mail: dawid.pilarski@panicsoftware.com (w celach zawodowych) oraz dawid.pilarski@pm.me (w celach prywatnych).